Project Progress Report
ICT 222-2

Phishing Detection System
WebGuard

Group No: 05

Registration Number	Name
UWU/ICT/23/004	K. A. N. P. Perera
UWU/ICT/23/005	L. V. C. S. U. Kumari
UWU/ICT/23/026	S. H. Thebuwana
UWU/ICT/23/080	A. W. D. H. T. Jayawickrama
UWU/ICT/23/081	S. A. N. Rathnasinghe

Supervised By:
Mr. V. Dishankan

Department of Information and Communication Technology

Faculty of Technological Studies

Uva Wellassa University

2023/2024 Batch

Table of Contents

LIST OF TABLES IV

LIST OF FIGURE V

CHAPTER 1: INTRODUCTION 1

1.1 PROJECT DESCRIPTION 1
1.2 BACKGROUND AND MOTIVATION 1
1.3 PROBLEM IN BRIEF 2
1.4 PROPOSED SOLUTION AND SIGNIFICANCE OF STUDY 2
1.5 OBJECTIVES 2
1.5.1 Main Objective 2
1.5.2 Specific Objectives 2
1.6 METHODOLOGY 3
1. Requirement Gathering 3
2. System Design 3
3. System Development 3
4. Testing 3
5. Deployment 3
6. Documentation 3

CHAPTER 2: REQUIREMENTS IDENTIFICATION 4

2.1 FUNCTIONAL AND NON-FUNCTIONAL REQUIREMENTS 4
2.1.1 Functional Requirements 4
2.1.2 Non-functional Requirements 4
2.2 USER LEVELS, USER ROLES 4
2.2.1 User 4
2.2.2 Administrator 4
2.3 SYSTEM REQUIREMENTS 5
2.3.1 Hardware Requirements 5
2.3.2 Software Requirements 6

CHAPTER 3: PROJECT DESIGN 7

3.1 UML DIAGRAMS 7
3.1.1 Use Case Diagram 7
3.1.2 Activity Diagram URL Scanning Process (USER) 8
3.1.3 Sequence Diagram 9

3.1.4 Class Diagram 10
3.1.5 MongoDB Schema Diagram 11
3.2 SYSTEM REQUIREMENTS 12
3.2.1 System Architecture Diagram 12
3.2.2 Component Diagram 13

CHAPTER 4: FUTURE PLAN 15

CHAPTER 5: PROJECT PLAN 16

5.1 INTRODUCTION 16
5.2 GANTT CHART 17
5.3 WORK BREAKDOWN STRUCTURE 17
5.4 MILESTONE 18

REFERENCES 19

APPENDIXES 20

APPENDIX A: USER INTERFACE 20
APPENDIX B – USER MANUAL 27
APPENDIX C: IMPORTANT SOURCE CODES 31
APPENDIX D: TEST RESULT 35

GROUP DETAILS 38

List of Tables

Table 1 : Hardware Requirements 5
Table 2: Software requirements 6
Table 3: Milestone 18
Table 4: Score table 32
Table 5: Safe result table 35
Table 6: Suspicious result table 36

List of Figure

Figure 1: Use case Diagram 7
Figure 2: Activity Diagram 8
Figure 3: Sequence Diagram 9
Figure 4: Class diagram 10
Figure 5: MongoDB Schema diagram 11
Figure 6: Architecture Diagram 12
Figure 7: Component Diagram 13
Figure 8: Gantt chart 17
Figure 9: Work Breakdown Structure 17
Figure 10: Home Screen 1 20
Figure 11: Home Screen 2 20
Figure 12: URL scan page 21
Figure 13: Scann History 21
Figure 14: Scan URL 22
Figure 15: Safe scan result 22
Figure 16: Scan URL(Suspicious) 23
Figure 17: Suspicious Scan result 23
Figure 18: Scan URL (Dangerous) 24
Figure 19: Dangerous Scan Result 24
Figure 20: Browser extension scan 25
Figure 21: Browser extension scan result 25
Figure 22: Admin Login page 26
Figure 23: Admin Dashboard 26
Figure 24: User Manual (Home Page) 27
Figure 25: User Manual (Scan Page) 28
Figure 26: User Manual (Result Page) 28
Figure 27: User Manual (Scan History) 29
Figure 28: User Manual(Browser extension) 29
Figure 29: User manual (Admin login) 30
Figure 30: User Manual (Admin dashboard) 30
Figure 31: Detection logic source code 31
Figure 32: Risk score calculation source code 32
Figure 33: MongoDB Connection source code 33
Figure 34: MongoDB collections 33
Figure 35: Browser extension manifest source code 34
Figure 36: Browser extension popup source code 34
Figure 37: Safe result page 35
Figure 38: Suspicious result page 36
Figure 39: Dangerous result table 37
Figure 40: Dangerous Result page 37

Chapter 1: Introduction

1.1 Project Description

The Phishing Detection System is a web-based cybersecurity application developed to help users identify potentially dangerous websites by analyzing website URLs. The system uses rule-based detection techniques to examine URL characteristics such as URL length, suspicious characters, suspicious domain patterns, suspicious keywords, HTTPS availability, and the use of IP addresses instead of domain names. Based on the analysis, the system generates a phishing risk score and classifies the website as **Safe**, **Suspicious**, or **Dangerous**.

The system provides users with the detection result, risk score, and a clear explanation of the reasons behind the generated result, enabling users to understand why a website is considered safe or potentially dangerous. In addition, the system includes cybersecurity awareness information to educate users about online threats and encourage safe browsing practices.

Users can enter a website URL for analysis, view scan analytics such as the total number of scanned URLs, the number of safe and dangerous websites, and recent scanning activities through graphical representations. The system also provides access to a limited scan history, where previously scanned URLs are partially masked to protect user privacy and security.

Furthermore, the system includes a browser extension that enables users to scan the currently visited website directly from their browser. An administrator login page is also provided, allowing authorized administrators to securely access administrative functions such as monitoring scanning activities, viewing system statistics, and managing system operations. All scan records are securely stored in a MongoDB database to support analysis and system monitoring.

1.2 Background and Motivation

The rapid growth of internet usage has significantly increased the number of cyber threats, with phishing attacks becoming one of the most common forms of online fraud. Phishing websites imitate legitimate websites to steal sensitive information such as usernames, passwords, banking credentials, and personal information. Many internet users find it difficult to distinguish between legitimate and fraudulent websites, making them vulnerable to cyberattacks.

Existing phishing detection solutions mainly rely on browser warnings, blacklists, or commercial cybersecurity services. However, these solutions often require paid subscriptions, technical knowledge, or enterprise-level infrastructure. Moreover, some systems provide warnings without explaining why a website is considered unsafe, reducing users' understanding of phishing threats.

The motivation for this project is to develop an affordable and user-friendly phishing detection system that performs real-time URL analysis and provides phishing risk scores together with clear reasons for the detection result. By integrating phishing detection into a browser extension, the system aims to improve cybersecurity awareness and help users make informed decisions before accessing potentially dangerous websites.

1.3 Problem in Brief

The number of phishing attacks is on the rise as cybercriminals develop fake-looking websites to obtain sensitive user info. With the evolution of phishing techniques, it can be difficult for many Internet users to recognize a dangerous site. While there are phishing detection solutions in place, many of them are blacklists, paid security services or browser warnings which contain only limited information about why a website is deemed unsafe. This means that users may not be aware of the dangers of visiting a suspicious site. Therefore, a user-friendly phishing detection system is needed to analyze website URLs, classify websites as Safe, Suspicious, or Dangerous, provide a risk score with clear reasons for the classification, and improve users' awareness of phishing threats.

1.4 Proposed Solution and Significance of Study

The proposed solution is a web-based Phishing Detection System that analyses website URLs using rule-based detection techniques to identify potentially dangerous websites. The system evaluates various URL characteristics and classifies websites as Safe, Suspicious, or Dangerous while generating a risk score and providing clear reasons for the detection result. It also includes a browser extension for convenient URL scanning, scan analytics to summarize scanning activities, cybersecurity awareness information to educate users, and an administrator portal for monitoring system activities.

Compared to many existing phishing detection solutions that mainly provide browser warnings or rely on blacklist-based detection, the proposed system offers additional features such as risk scoring, understandable reasons for detection, scan analytics, and cybersecurity awareness information. These features help users better understand website security risks and improve their awareness of phishing attacks, enabling them to make safer decisions while browsing the internet.

1.5 Objectives

1.5.1 Main Objective

To develop a web-based Phishing Detection System with a browser extension that analyzes website URLs and helps users identify potentially dangerous or suspicious websites.

1.5.2 Specific Objectives

- To develop a browser extension for scanning the currently visited website.
- To analyses website URLs using rule-based detection techniques.
- To classify websites as **Safe**, **Suspicious**, or **Dangerous** based on the analysis.
- To generate a phishing risk score and provide clear reasons for the detection result.
- To provide scan analytics and a secure scan history for users.
- To provide cybersecurity awareness information to improve users' awareness of online threats.
- To develop an administrator portal for monitoring system activities and scan records.

1.6 Methodology

The development of the Phishing Detection System follows the Agile Software Development Methodology, which supports iterative development, continuous testing, and regular feedback throughout the software development life cycle.
The software development life cycle consists of the following phases:

1. **Requirement Gathering**

   - Conducted a literature review on phishing detection techniques and existing systems.
   - Identified the functional and non-functional requirements.
   - Gathered user and system requirements through research and discussions.

2. **System Design**

   - Designed the overall system architecture.
   - Created the database structure and user interface designs.
   - Prepared UML diagrams, including use case, activity, sequence, class, component, and MongoDB collection schema diagrams.

3. **System Development**

   - Developed the frontend using HTML, CSS, and JavaScript.
   - Implemented the backend using Python.
   - Integrated the frontend, backend, and MongoDB database.
   - Developed and integrated the browser extension for URL scanning.

4. **Testing**

   - Functional and integration testing were completed to verify the implemented features.
   - Bug fixing and performance improvements are currently being carried out to enhance the system's reliability and stability.

5. **Deployment**

   - The web application and browser extension will be deployed after the completion of bug fixing and final validation.
   - Final system configuration and deployment activities will be carried out before project completion.

6. **Documentation**

   - The Project Proposal and Software Requirement Specification (SRS) were prepared during the initial stages of the project.
   - The Project Progress Report is being prepared to document the current implementation progress.
   - The User Manual and Final Project Report will be completed before the final submission.

Chapter 2: Requirements Identification

2.1 Functional and Non-functional Requirements

2.1.1 Functional Requirements

The functional requirements describe the main functions that the Phishing Detection System performs to meet user requirements. The system allows users to enter a website URL for analysis through the web application or scan the currently visited website using the browser extension. It analyses the URL using rule-based detection techniques and generates a phishing risk score with the detection result classified as Safe, Suspicious, or Dangerous. The system also provides clear reasons for the generated result and cybersecurity awareness information to help users understand potential online threats. Additionally, users can view scan analytics, while administrators can securely log in to monitor scanning activities and system statistics. The system also provides a limited scan history.

2.1.2 Non-functional Requirements

The system is designed to provide good performance, security, reliability, usability, and maintainability. It should analyses URLs efficiently and display results within a short period. The administrator portal is protected through authentication to prevent unauthorized access. The user interface is designed to be simple and easy to use for users with different technical backgrounds. The system is also designed using a modular architecture to support future enhancements and maintenance.

2.2 User levels, User roles

2.2.1 User

The user is the primary user of the system and does not require an account or login to access the web application. Users can:

- Enter a website URL for analysis.
- Scan the currently visited website using the browser extension.
- View the detection result.
- View the phishing risk score and reasons for the result.
- View scan analytics.
- Access cybersecurity awareness information.

2.2.2 Administrator

The administrator has secure access to the administrative portal through a login page. The administrator is responsible for monitoring system activities and viewing overall scanning statistics. Administrator responsibilities include:

- Logging into the administrator portal.
- Monitoring scanned URLs.
- Viewing scan analytics and system statistics.
- Monitoring recent scanning activities.

2.3 System requirements

2.3.1 Hardware Requirements

Table 1 : Hardware Requirements

Component	**Minimum Requirement**
Processor	Intel Core i3 / Apple M1 or equivalent
RAM	4 GB (8 GB Recommended)
Storage	10 GB Available Space
Internet Connection	Required
Display	1366 × 768 Resolution or Higher

The selected hardware requirements are sufficient to support the development, testing, and execution of the web application, browser extension, and MongoDB database while providing acceptable system performance.

2.3.2 Software Requirements

Table 2: Software requirements

Software / Technology	**Purpose**	**Justification**
Python (Flask)	Backend Development	Selected for implementing the phishing detection logic and handling server-side operations efficiently.
HTML	Frontend Development	Used to create the structure of the web application.
CSS	User Interface Design	Used to create a responsive and user-friendly interface.
JavaScript	Frontend Functionality	Enables interactive features and communication with the backend.
MongoDB	Database Management	Stores scan records, analytics data, and system information efficiently.
Google Chrome	Browser Extension	Supports the development and execution of the browser extension for URL scanning.
Visual Studio Code	Development Environment	Provides an efficient environment for coding, debugging, and project development.
Git & GitHub	Version Control	Used to manage source code, track changes, and support collaborative development.
Figma	UIUX designs	Used to design the user interface prototypes and improve the user experience before implementation.

Chapter 3: Project Design

3.1 UML Diagrams

3.1.1 Use Case Diagram

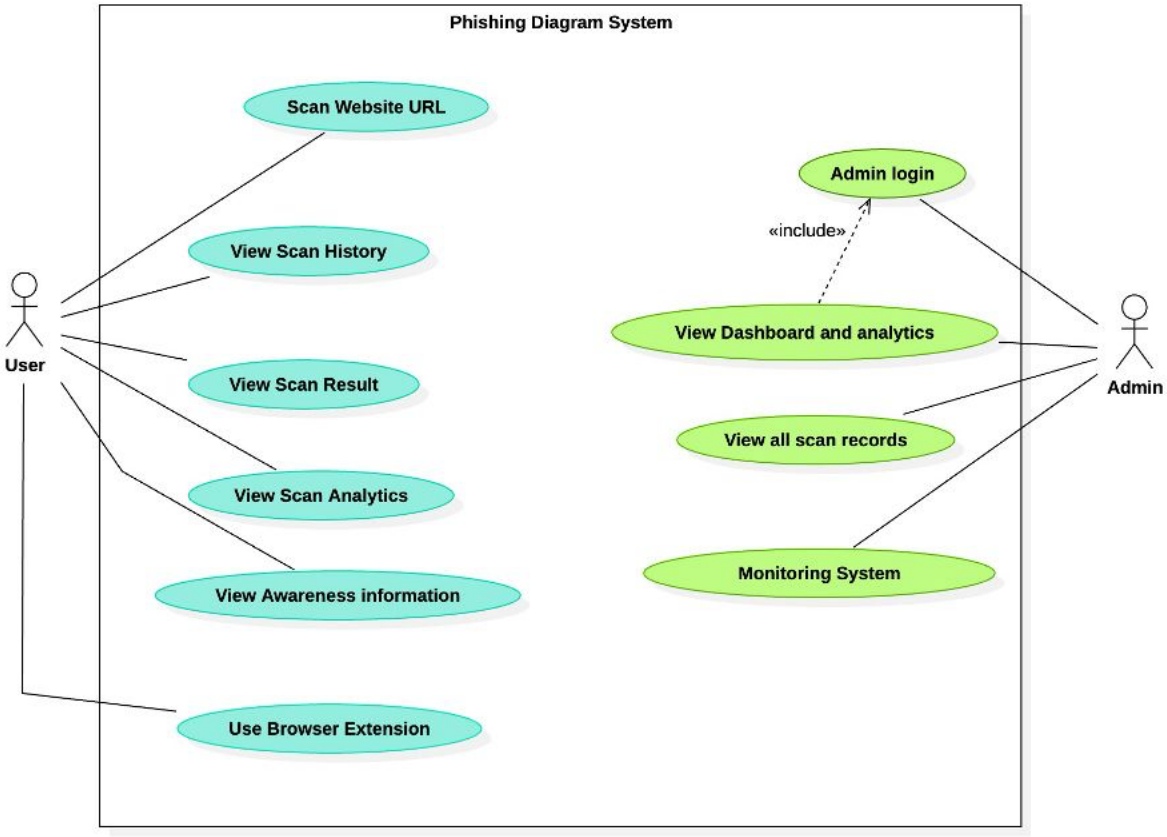


Figure 1: Use case Diagram

The Use Case Diagram illustrates the interaction between the actors and the WebGuard system. The main actors are the User and the Administrator. The User can enter URLs, scan websites, view detection results, view phishing risk scores, access previous scan history, and use the browser extension for quick URL scanning. The Administrator can log in to the admin dashboard, monitor scan statistics, review stored scan results, and manage system monitoring activities. The diagram shows the functional services provided by the system and how each actor interacts with those services.

3.1.2 Activity Diagram URL Scanning Process (USER)

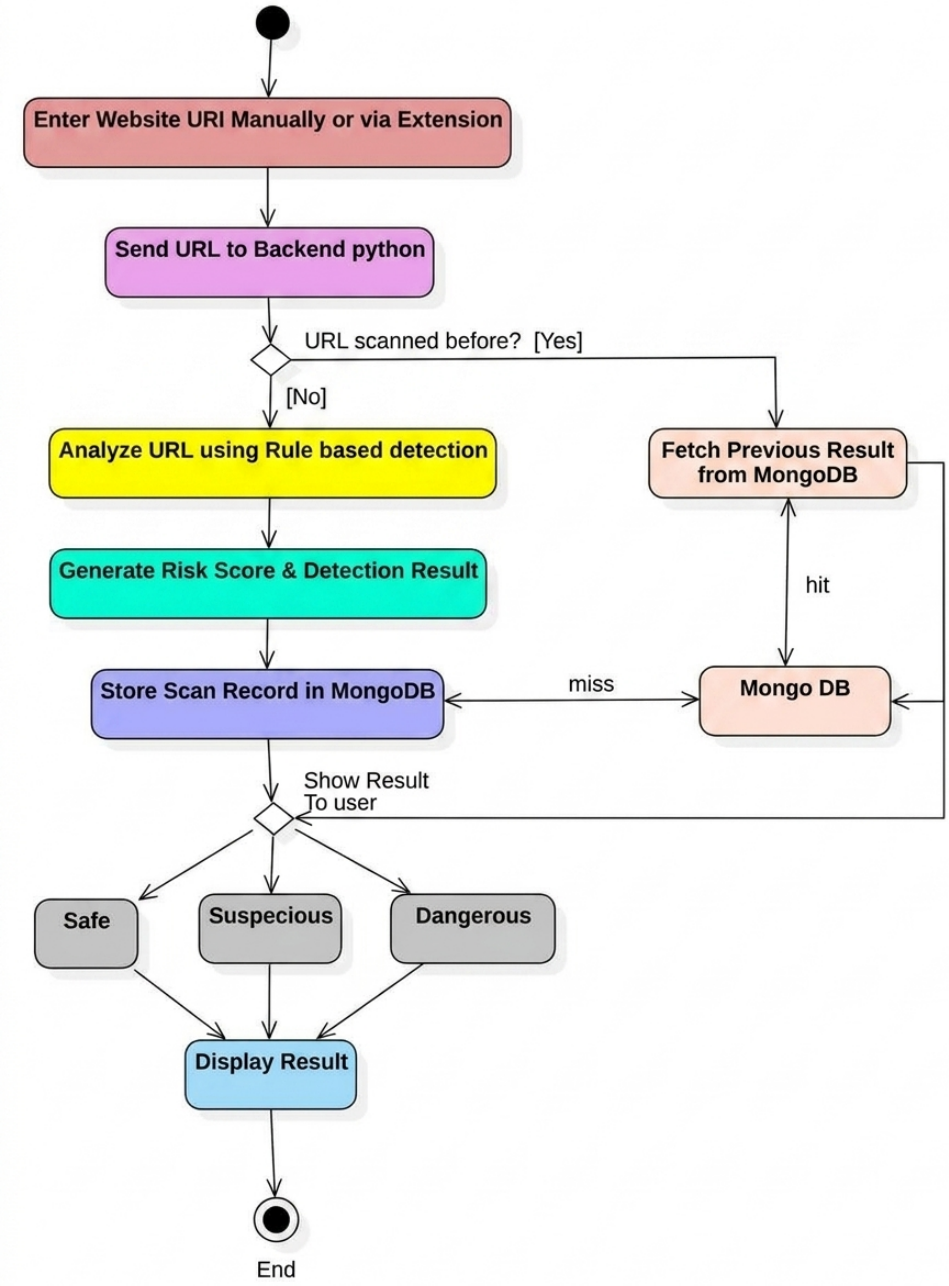


Figure 2: Activity Diagram

The activity diagram represents the workflow of the WebGuard URL Safety Check System. The process begins when the user enters a URL manually or through the browser extension. The URL is sent to the Python backend, which checks MongoDB for an existing scan record.

If the URL exists, the stored result is retrieved directly. If not, the URL is analyzed by the rule-based detection engine, which calculates the risk score, assigns a status (Safe, Suspicious, or Dangerous), and stores the new result in MongoDB.

Finally, the system displays the scan result with the risk score and reason on the user dashboard

3.1.3 Sequence Diagram

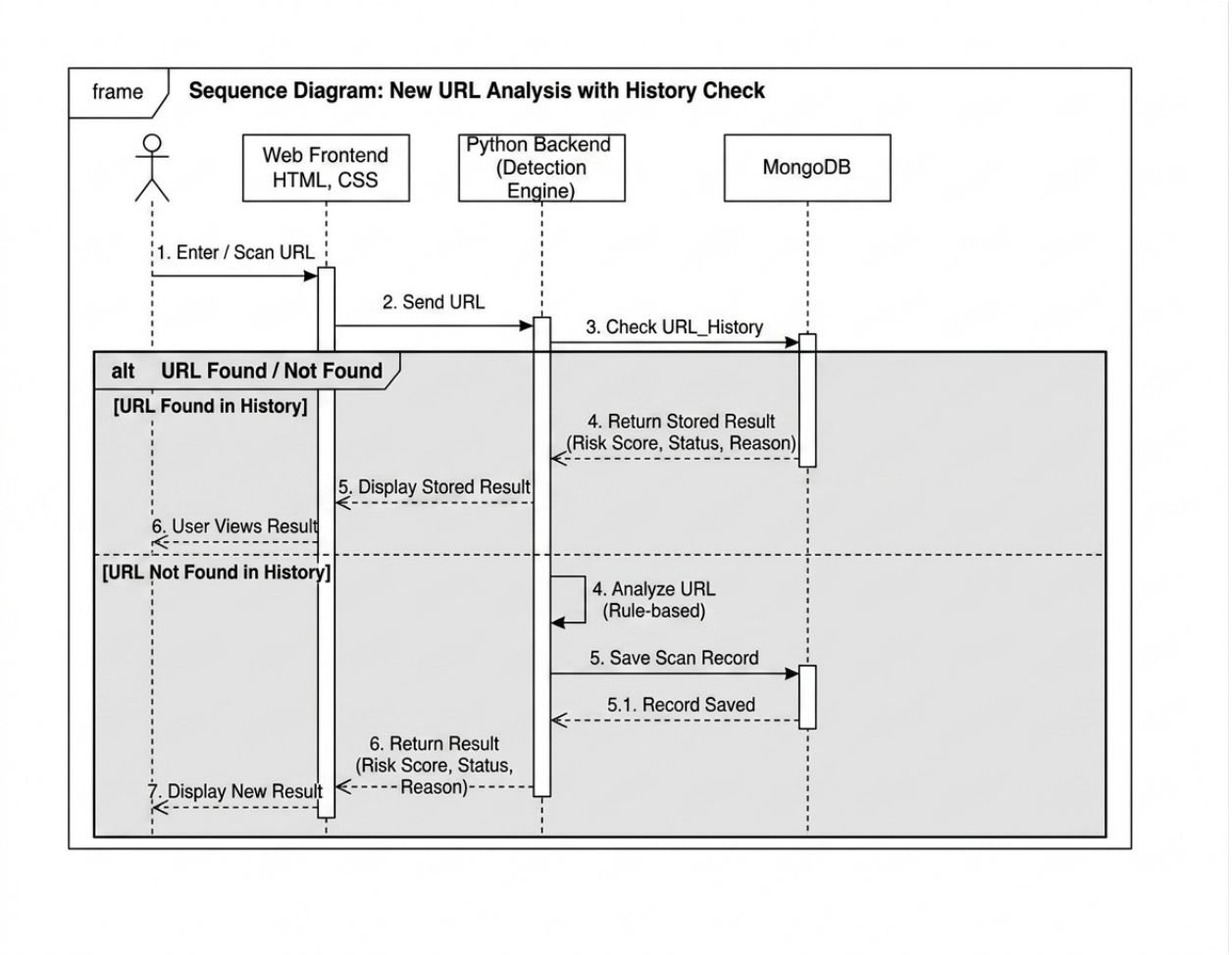


Figure 3: Sequence Diagram

The sequence diagram illustrates the interaction flow between the User, Web Interface/Browser Extension, Python Backend API, URL Analysis Engine, and MongoDB database. When a user submits a URL, the system first checks whether the URL exists in the scan history. If a previous result is available, the stored result is returned without re-analysis. Otherwise, the URL is analyzed by the detection engine, a risk score and classification are generated, and the result is stored in MongoDB before being displayed to the user.

3.1.4 Class Diagram

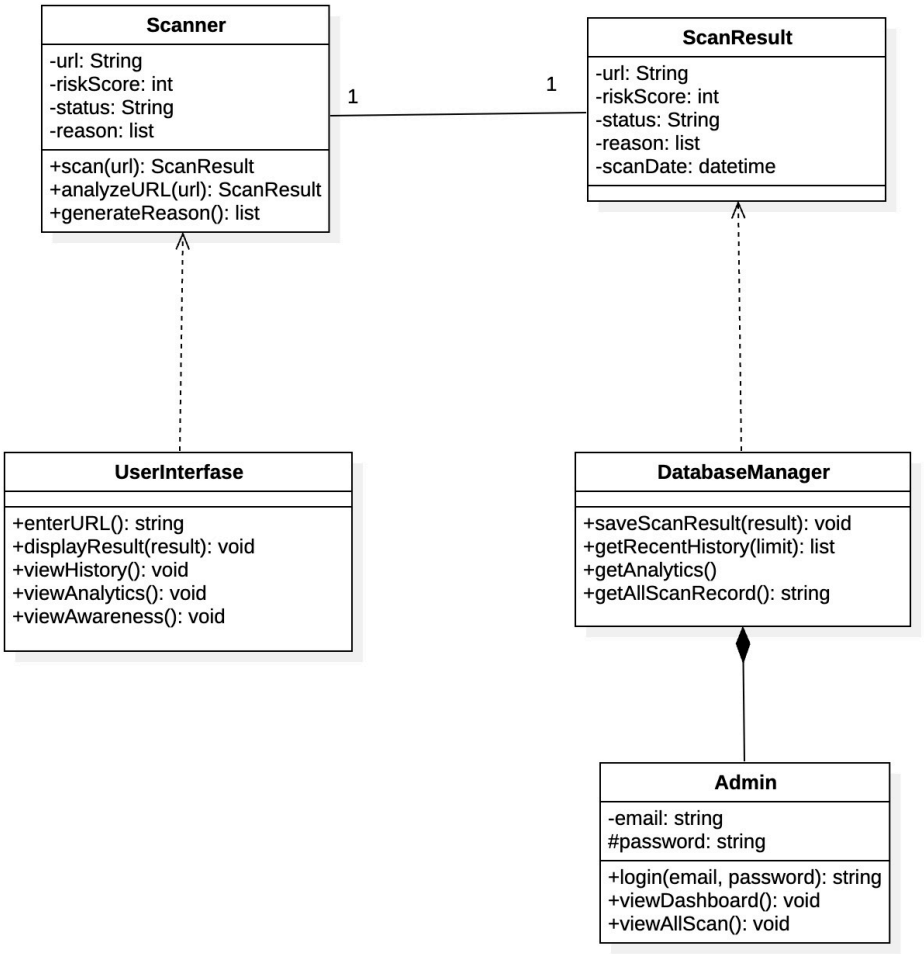


Figure 4: Class diagram

This class diagram represents the main classes of the system and their relationships, including URL analysis, risk scoring, scan history management, and administrator management

3.1.5 MongoDB Schema Diagram



The MongoDB Schema Diagram shows the structure of the collections used in the WebGuard system. The main collection is scans (URL_History), which stores the scanned URL, risk score, detection status, explanation, scan date, and a masked URL for secure history display.

Before performing a new analysis, the backend checks this collection to determine whether the URL has already been scanned. If a matching record is found, the system reuses the stored result and returns it immediately without running the detection process again. This mechanism improves response time and reduces unnecessary processing.

The admins collection stores administrator login credentials and roles for secure access to the admin dashboard. The schema is designed to support scan history, result reuse, analytics, and administrative monitoring features.

Figure 5: MongoDB Schema diagram

Admin Collection	URL_History Collection
• Stored administrator login credentials • Used for admin authentication • Only admin can access system monitoring and all scan records.	• Stored all scanned URL records • Contains risk score, status, reasons and scan data • Used for analytics, recent history and admin dashboard. • Users can view limited recent history.

3.2 System requirements

3.2.1 System Architecture Diagram

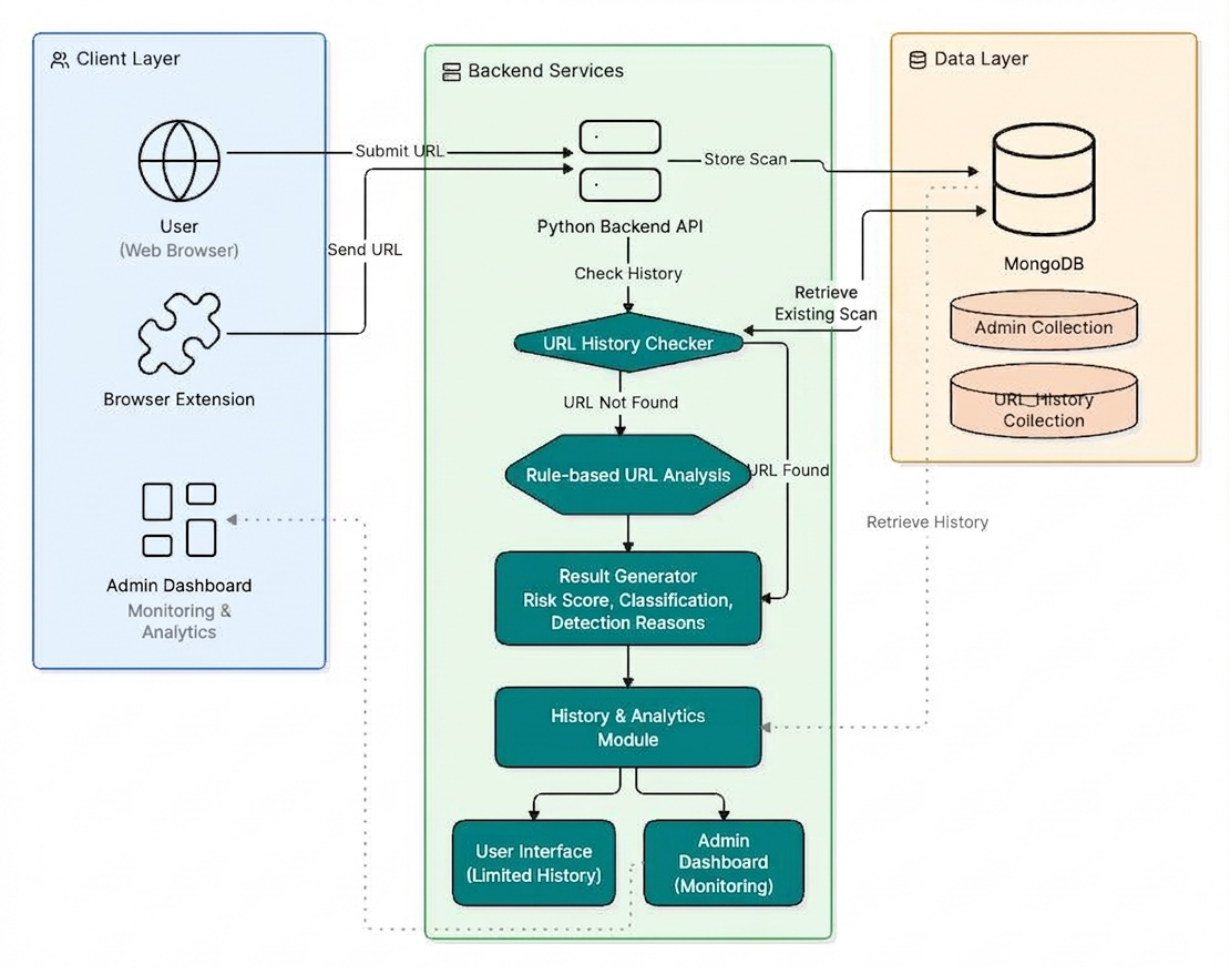


Figure 6: Architecture Diagram

3-Tier System Architecture of WebGuard

1. Client Layer (Frontend)

- Web Interface: Allows users to enter URLs and view scan results.
- Browser Extension: Captures active URLs for quick scanning.
- Admin Dashboard: Displays scan statistics and monitoring information.

2. Backend Layer (Processing)

- Python Backend API: Handles requests from users and the extension.
- URL Analysis Engine: Performs rule-based URL security analysis.
- Result Generator: Generates risk scores, classifications (Safe, Suspicious, Dangerous), and detection reasons.
- History & Analytics Module: Manages scan records and statistics.

3. Data Layer (Database)

- MongoDB: Stores system data.
- Admin Collection: Stores administrator account details.
- URL_History Collection: Stores scanned URLs, scores, statuses, reasons, and timestamps.

3.2.2 Component Diagram

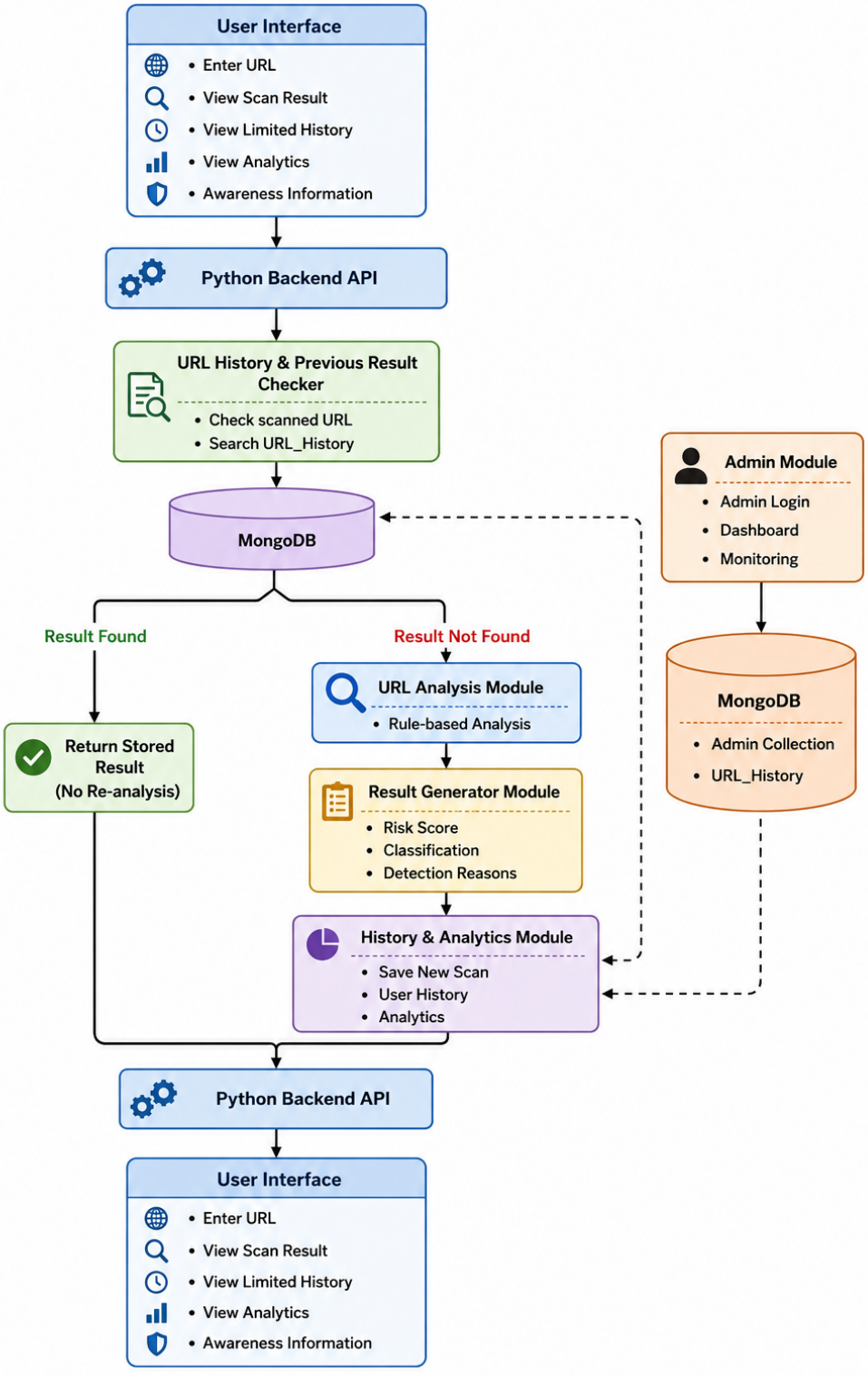


Figure 7: Component Diagram

The Component Diagram illustrates the modular architecture of the WebGuard system and shows communication between client, backend, database, and management components.

1. Client Layer

- User Interface: Allows users to submit URLs and view scan results, history, and analytics.
- Browser Extension: Captures active URLs and sends them for real-time scanning.

2. Backend Layer

- Python Backend API: Acts as the main controller that manages requests and communication between components.
- URL History Checker: Checks whether the submitted URL already exists in the database.
- If a previous result exists, the system reuses the stored scan result and returns it directly without performing a new analysis.
- If no previous result exists, the URL is forwarded to the detection engine for analysis.
- URL Analysis Module: Performs rule-based URL security analysis for new URLs.
- Result Generator: Generates risk scores, classifications (Safe, Suspicious, Dangerous), and detection reasons.
- History & Analytics Module: Stores new scan results and updates system statistics.

3. Data Layer

- MongoDB: Provides persistent data storage.
- URL_History Collection: Stores scanned URLs, risk scores, statuses, reasons, and timestamps for future reuse.
- Admin Collection: Stores administrator authentication details.

4. Admin Layer

- Admin Module: Provides secure login, dashboard access, and monitoring of scan statistics.

Chapter 4: Future Plan

The proposed Phishing Detection System can be further enhanced with the following features:

- Implement machine learning-based phishing prediction to improve detection accuracy.
- Add QR code phishing detection to identify malicious QR codes.
- Develop an email phishing analysis feature to detect phishing emails.
- Develop a mobile application for Android and iOS devices.
- Integrate advanced threat intelligence services to improve real-time threat detection.
- Implement screenshot-based phishing analysis using image recognition techniques.
- Support multilingual awareness content for users.
- Expand browser extension support to browsers such as Microsoft Edge and Mozilla Firefox.

These are excellent future enhancements because they go beyond your current scope.

Chapter 5: Project Plan

5.1 Introduction

The WebGuard Phishing Detection System follows a structured implementation plan divided into planning, development, testing, optimization, and deployment phases. The project activities were organized according to the Work Breakdown Structure (WBS), covering requirement analysis, system design, frontend development, backend implementation, testing, optimization, and final documentation.

Completed Activities
During the planning phase, the team completed requirement gathering, literature review, proposal preparation, system analysis, architecture design, database design, and user interface design. The system scope includes URL analysis, phishing detection, browser extension support, database storage, scan history, and administrator monitoring.
During the development phase, the frontend interface, Python backend API, database integration, browser extension, and administrator dashboard components were implemented. The Python backend was developed to handle URL feature extraction, rule-based phishing analysis, risk score calculation, Safe/Suspicious/Dangerous classification, detection result generation, and database communication.
The administrator dashboard was implemented to monitor scan statistics, review detection results, and manage system information.

Current Activities
The project is currently in the testing and optimization phase. Integration testing is being performed between the frontend interface, Python backend, database, and browser extension components. Bug fixing, system validation, and performance improvements are ongoing.
The team is also improving detection reliability by refining existing URL analysis rules and validating generated risk scores. Additional phishing detection rules, such as enhanced suspicious keyword detection and improved suspicious pattern identification, are planned for future improvements.

Remaining Activities
During the final phase, the team will focus on final system optimization, deployment preparation, documentation completion, user manual preparation, and final report and presentation activities.
Future enhancements will include expanding the detection rules, improving scan security features such as secure history viewing, and introducing advanced phishing analysis techniques.

5.2 Gantt Chart

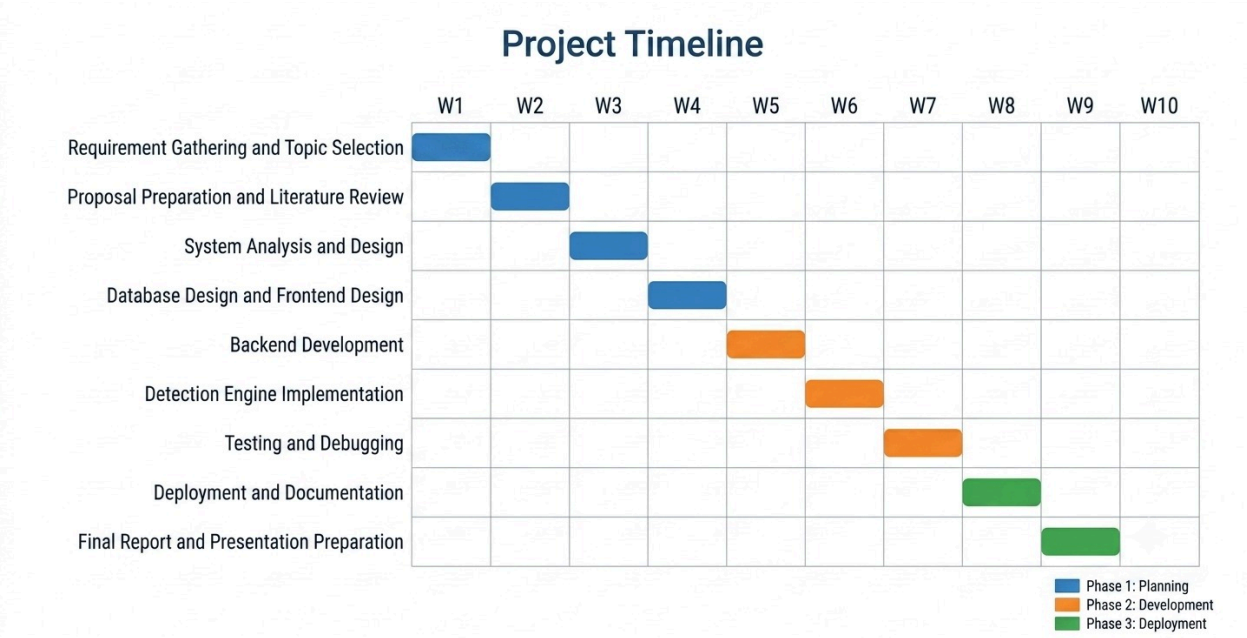


Figure 8: Gantt chart

5.3 Work Breakdown Structure

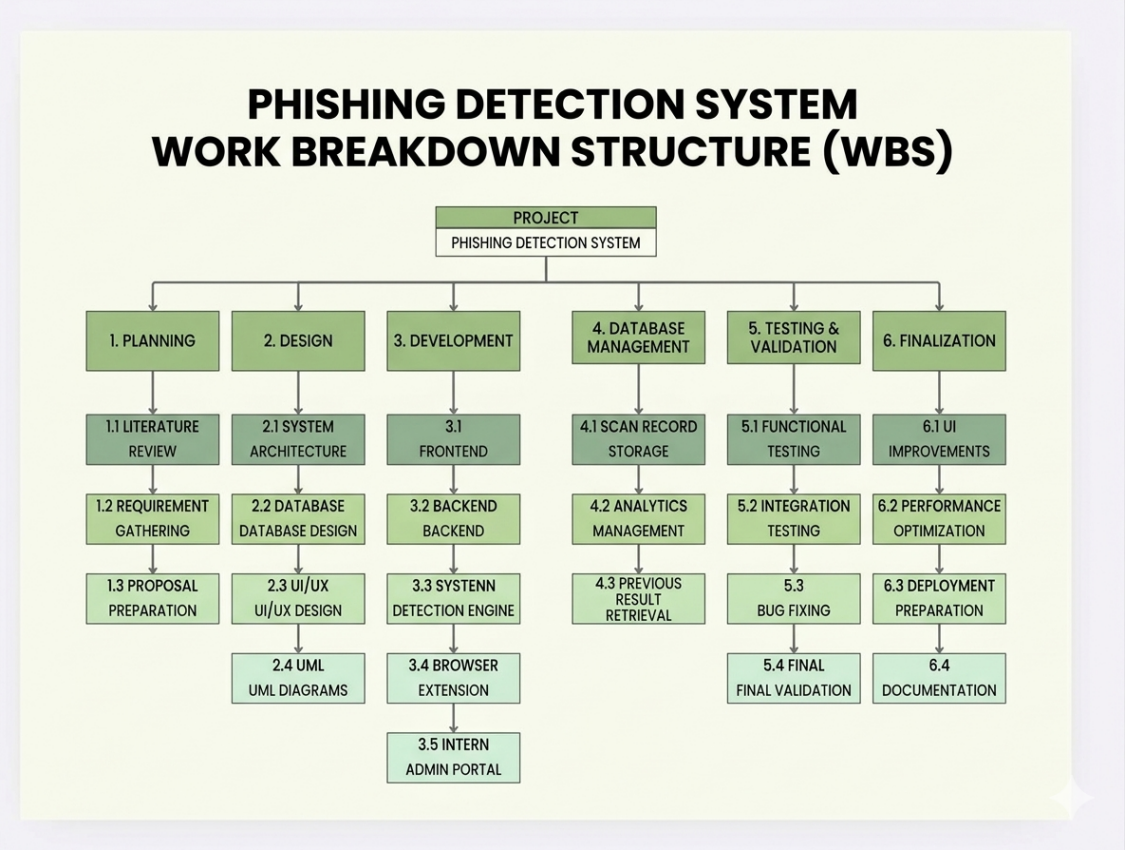


Figure 9: Work Breakdown Structure

5.4 Milestone

Table 3: Milestone

Milestone	Status
Project Proposal	Completed
Requirement analysis	Completed
System design	Completed
Frontend development	Completed
Backend development	Completed
Browser extension development	Completed
Integration Testing	Successfully ongoing
Bug Fixing and Final Validation	In Progress
Deployment	Pending
Documentation	In progress

References

[1] S. Kumar and R. Singh, “A Survey on Phishing Detection Techniques,” IEEE Access, vol. 10, pp. 98765–98780, 2022.

[2] Y. Li and H. Zhang, “Real-Time Browser Extension for Phishing Website Detection,” IEEE Access, vol. 12, pp. 112233–112245, 2025.

[3] Python Software Foundation, “Python Documentation.”
Available: https://docs.python.org/

[4] Pallets Projects, “Flask Web Framework Documentation,”
Available: https://flask.palletsprojects.com/

[5] MongoDB Inc., “MongoDB Documentation.”
Available: https://www.mongodb.com/docs/

[6] Google Chrome Developers, “Chrome Extensions Documentation.”
Available: https://developer.chrome.com/docs/extensions/

[7] OWASP Foundation, “OWASP Top 10 Web Application Security Risks.”
Available: https://owasp.org/www-project-top-ten/

[8] GitHub Inc., “GitHub Documentation,”
Available: https://docs.github.com/

Appendixes

[Any detailed description, user manual, selected important source codes; set of data, Images of GUI (Screenshots) could be included under this. Appendices must be named in alphabetical order (Appendix A, Appendix B…), and also give a name for each appendix.]

Appendix A: User Interface

A.1 Home Page

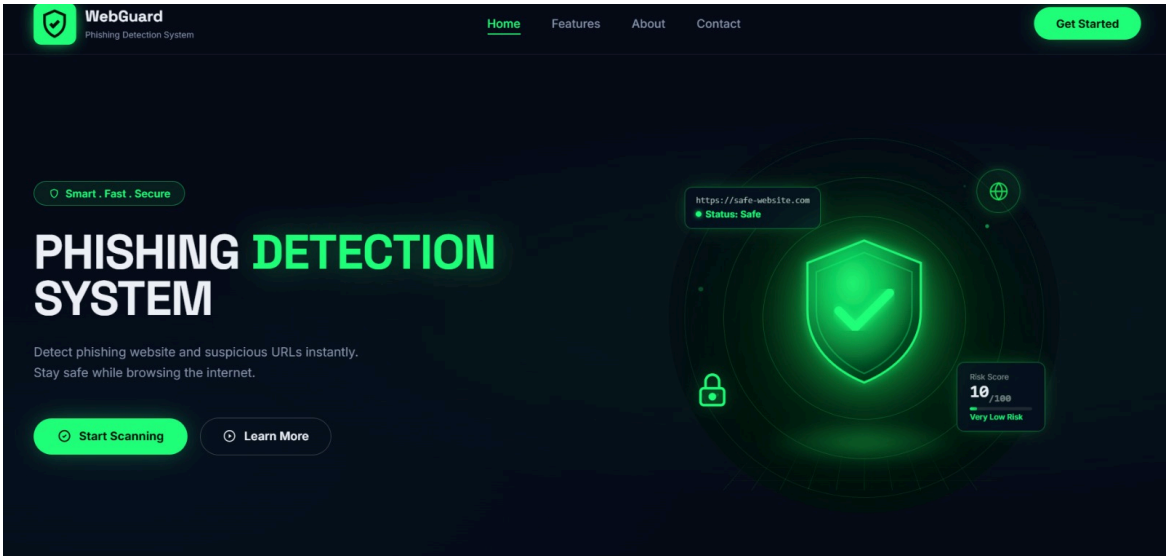


Figure 10: Home Screen 1

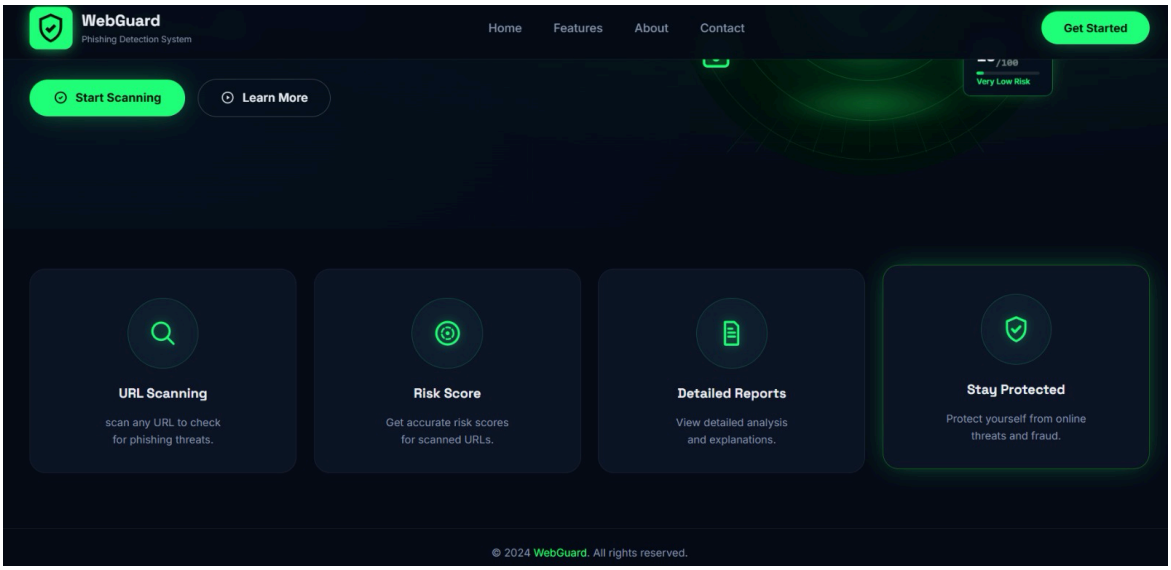


Figure 11: Home Screen 2

A.2 URL Scan Page

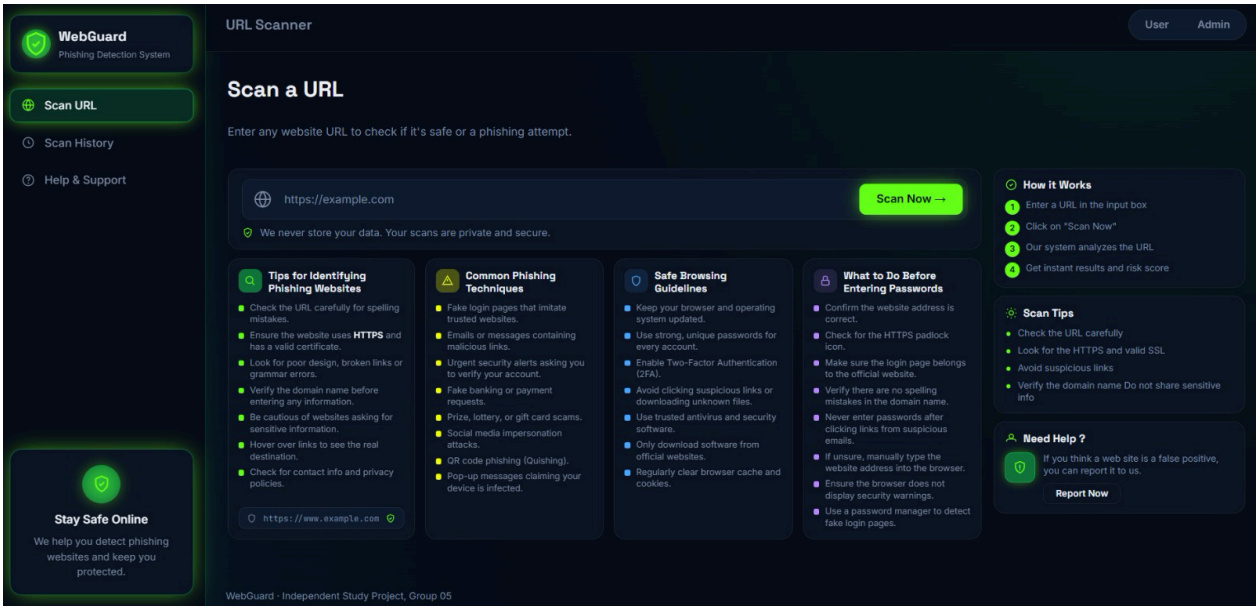


Figure 12: URL scan page

A.3 Scan Analytics dashboard for users

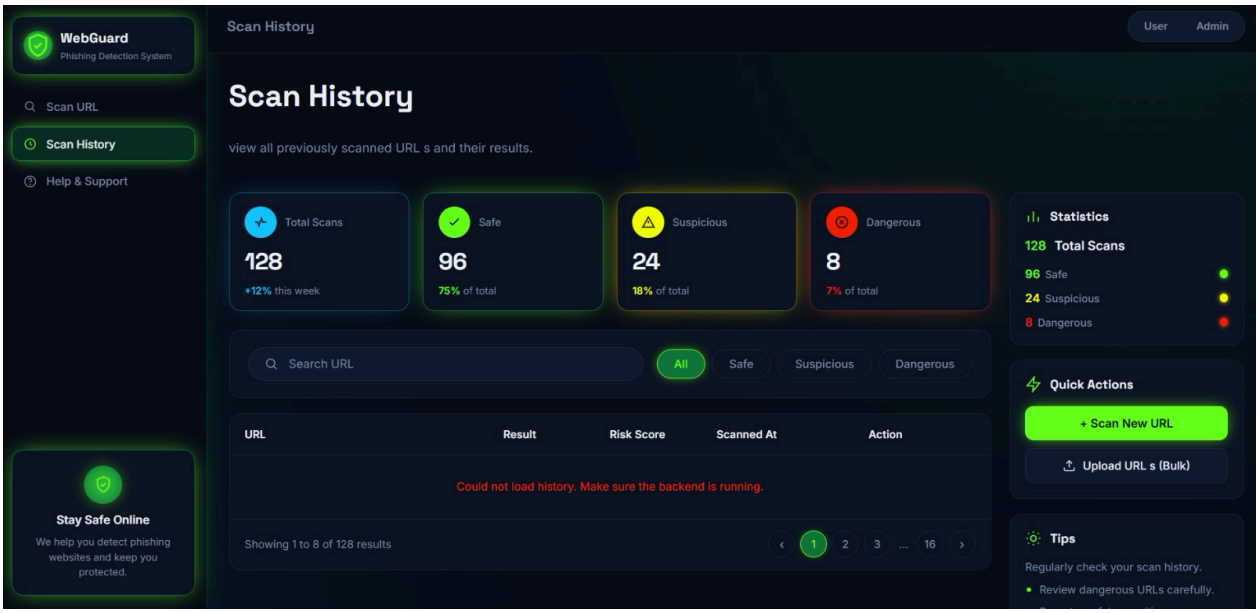


Figure 13: Scann History

A.4 Scan Result Page

Safe Webpage

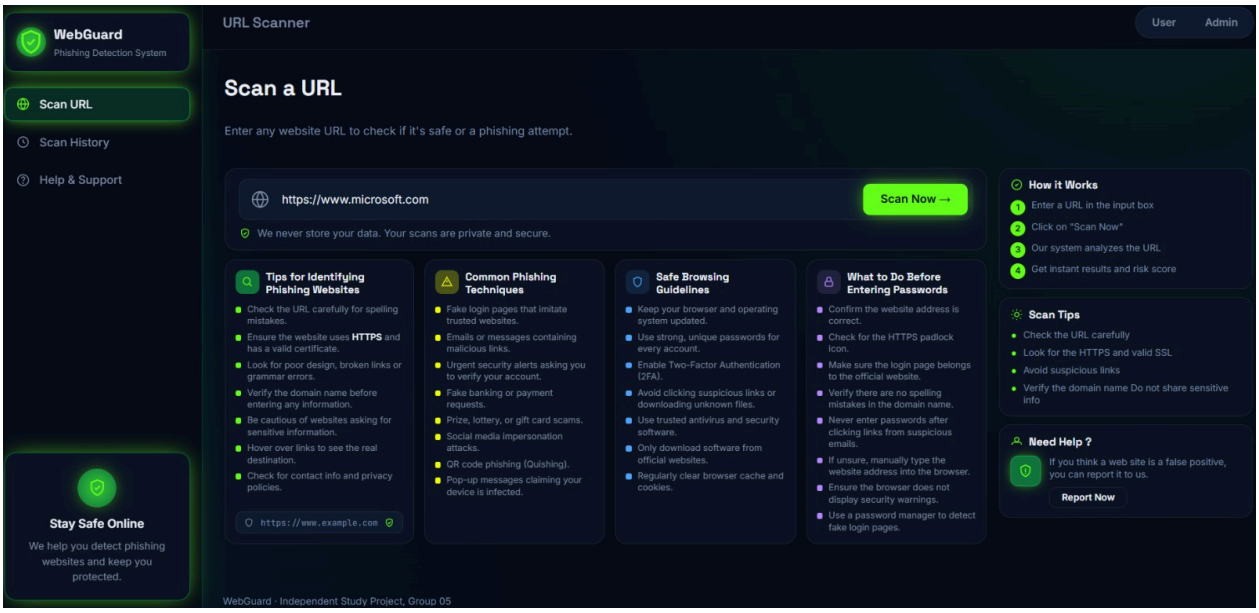


Figure 14: Scan URL

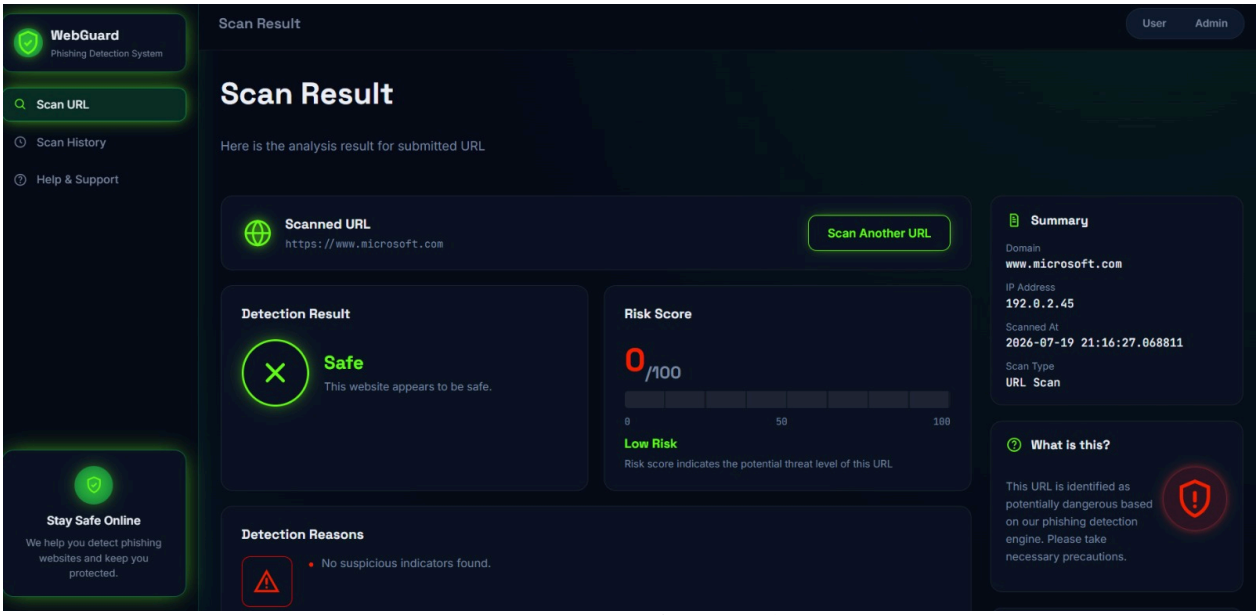


Figure 15: Safe scan result

Suspicious webpage

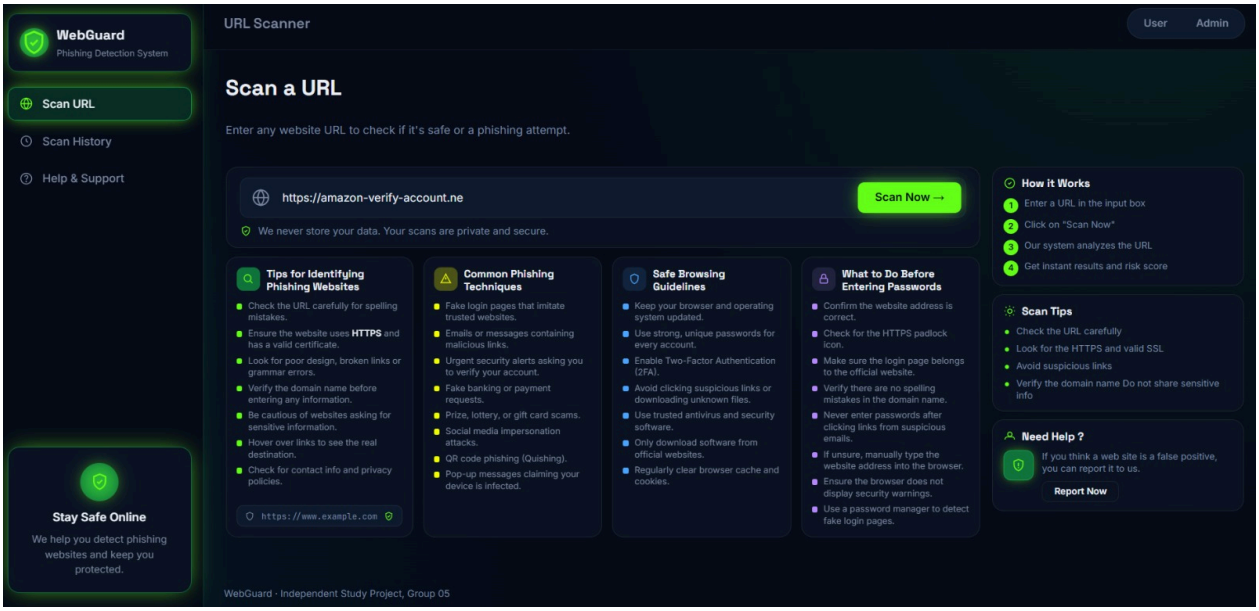


Figure 16: Scan URL(Suspicious)

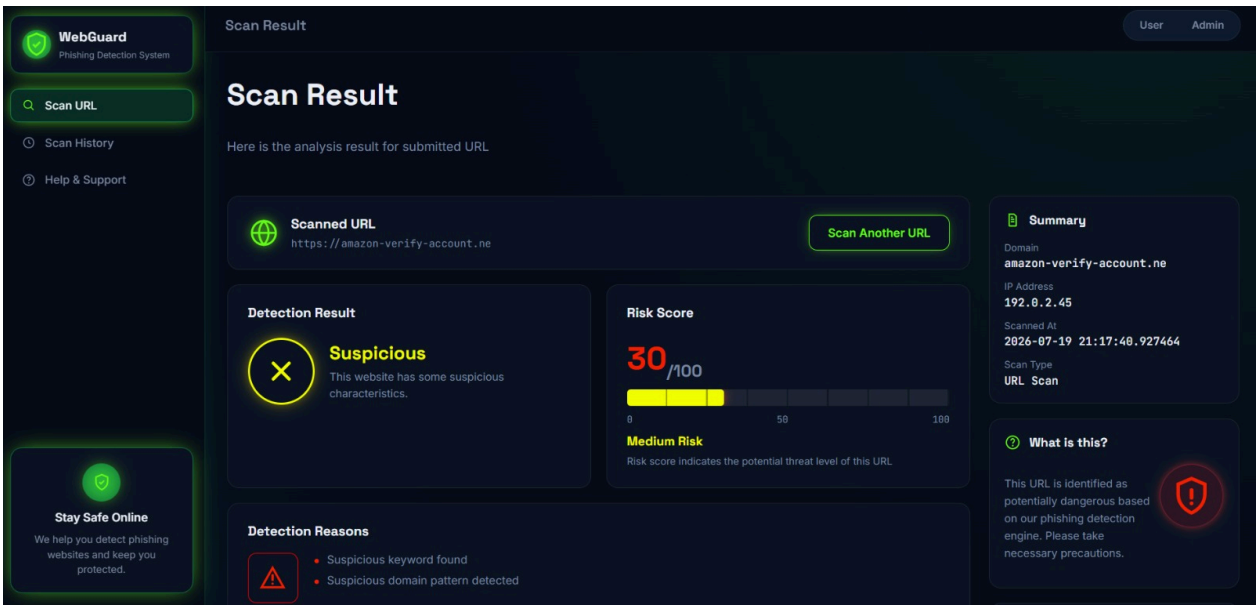


Figure 17: Suspicious Scan result

Dangerous webpage

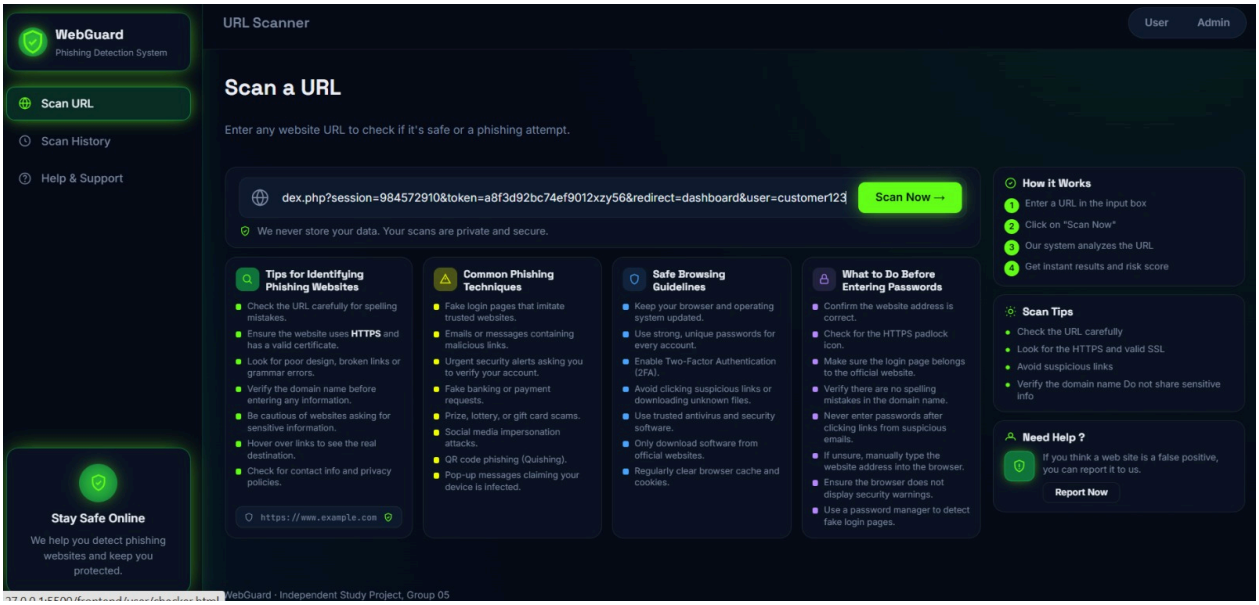


Figure 18: Scan URL (Dangerous)

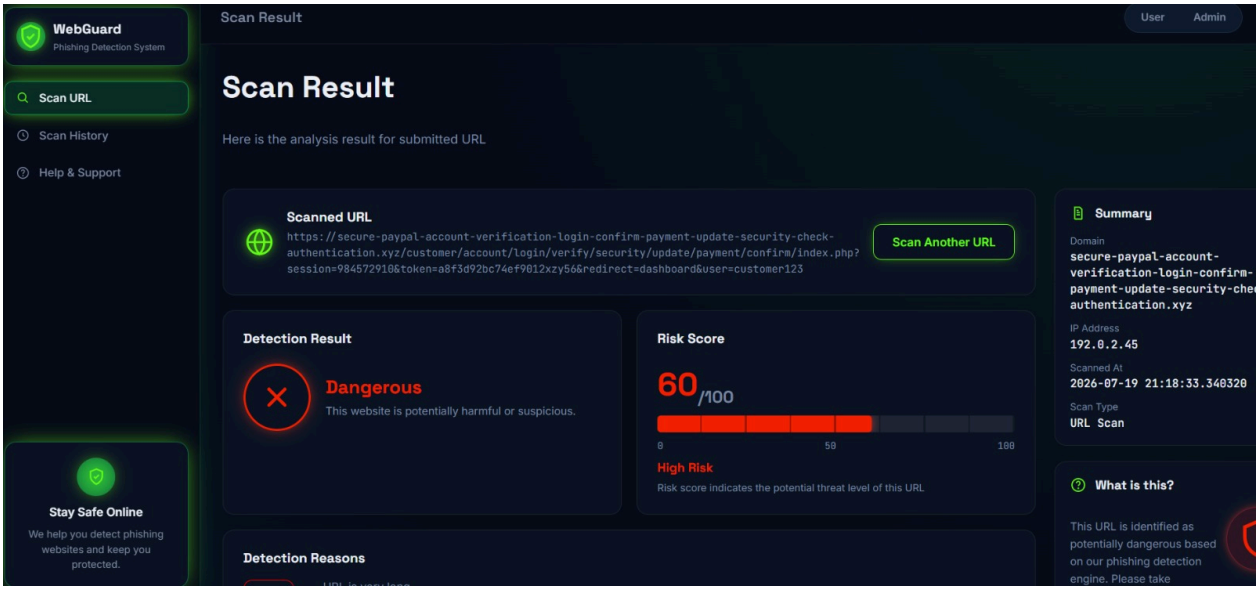


Figure 19: Dangerous Scan Result

A.5 Browser Extension Popup

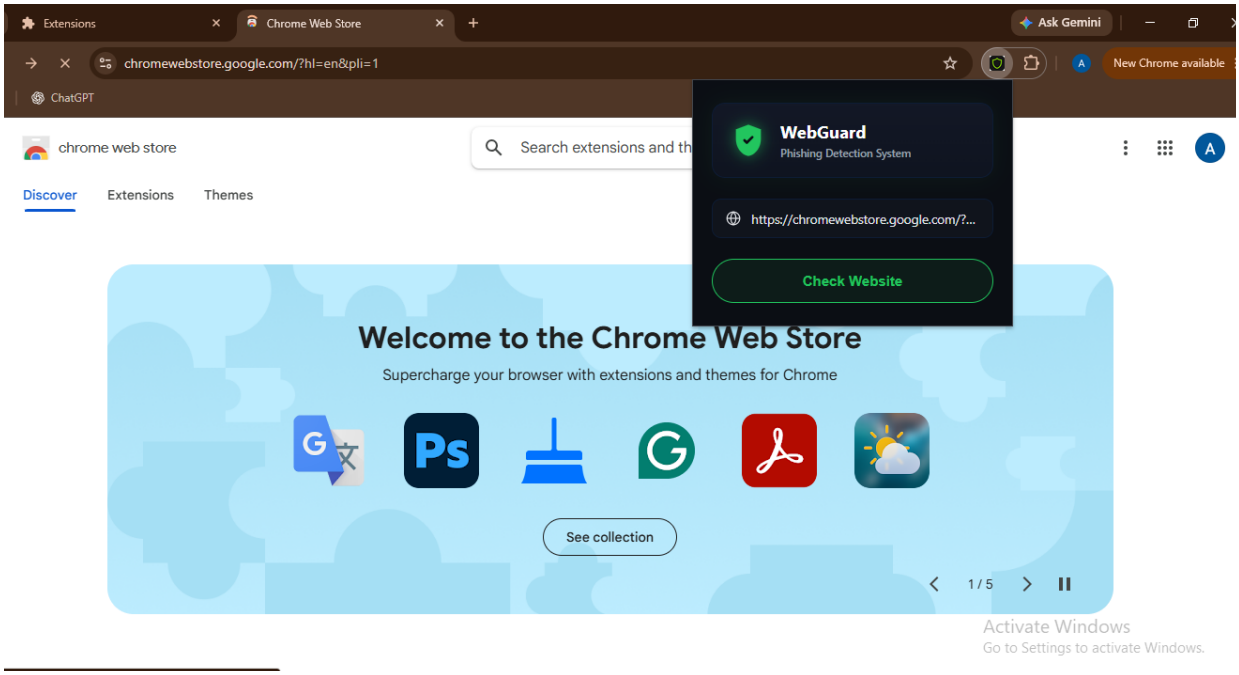


Figure 20: Browser extension scan

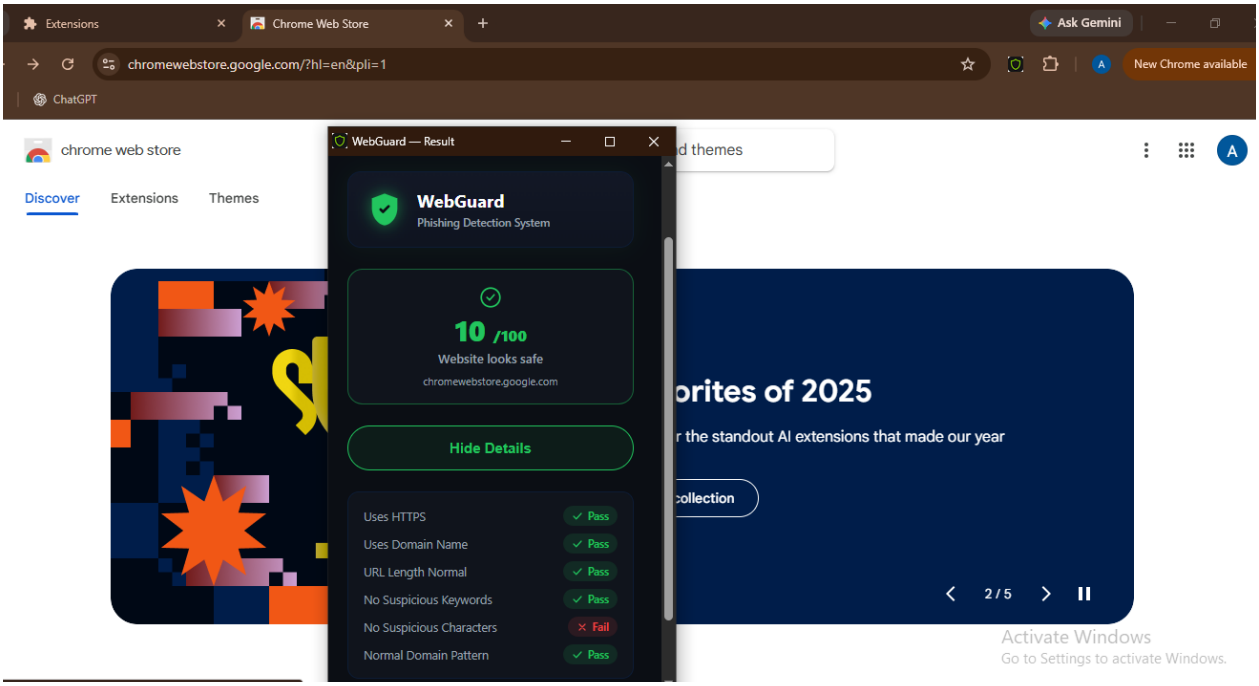


Figure 21: Browser extension scan result

A.6 Admin Login Page

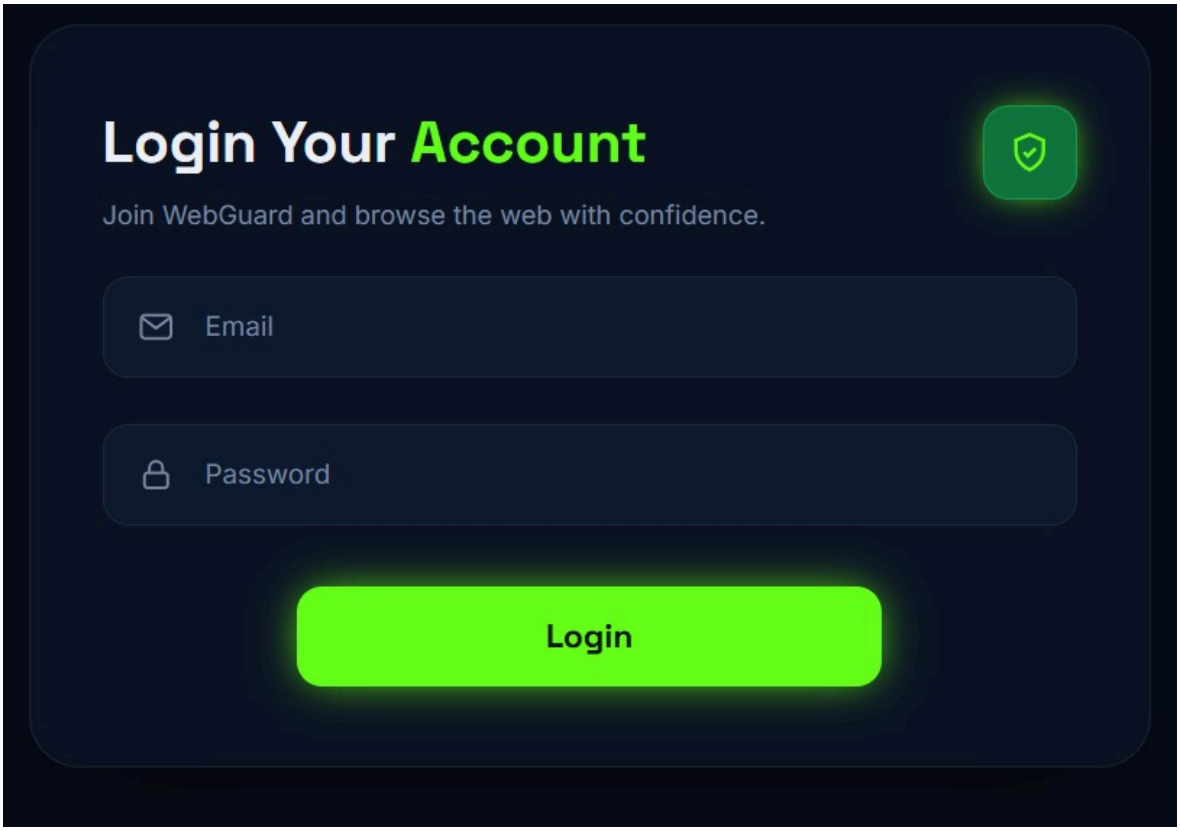


Figure 22: Admin Login page

A.7 Admin Dashboard

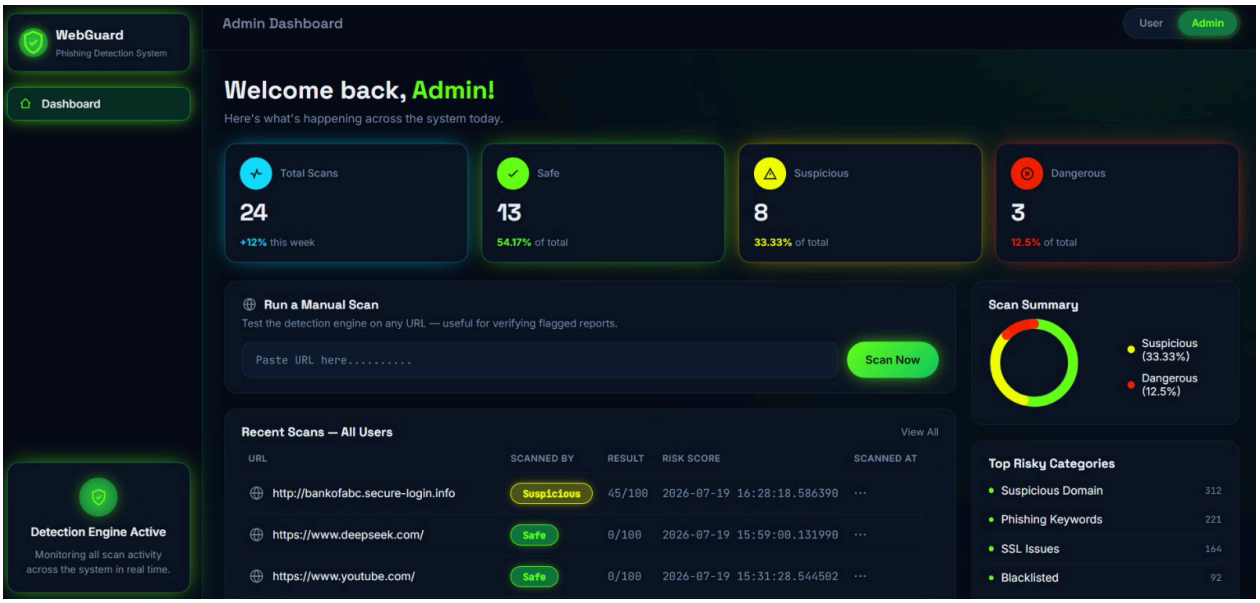


Figure 23: Admin Dashboard

Appendix B – User Manual

B.1 Introduction

The phishing Detection System is a web-based application developed to help users identify potentially dangerous websites by analyzing website URLs. Users can scan URLs, view risk scores and detection reasons, check scan analytics and use the browser extension to scan the currently visits website. This user manual provides basic instructions for operating the system.

B.2 Home page

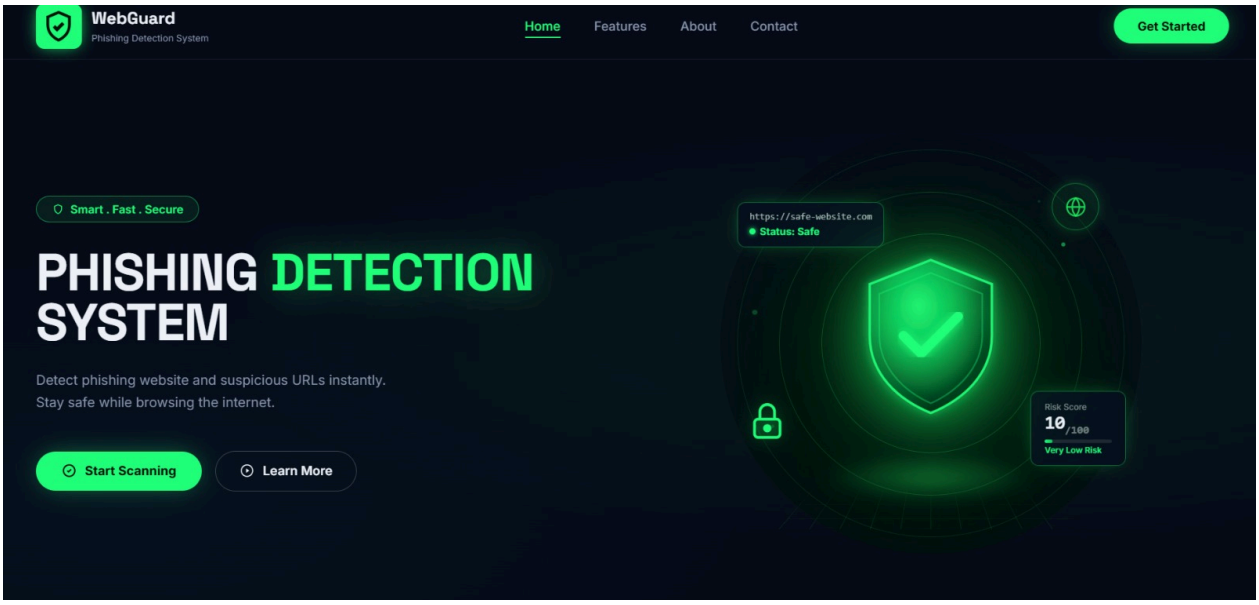


Figure 24: User Manual (Home Page)

The home page is the main entry point of the system. It introduces the phishing detection platform, providing global navigation link, and features a prominent “Start scanning” action. From this single interface, user can immediately input a URL to trigger the backend scanner and view instant risk metrics.

B.3 URL scan Page

The URL scan page allow user to enter a website URL and initiate the scanning process. After clicking the Scan URL button, the system sends the URL to the backend server for analysis using rule-based detection techniques.

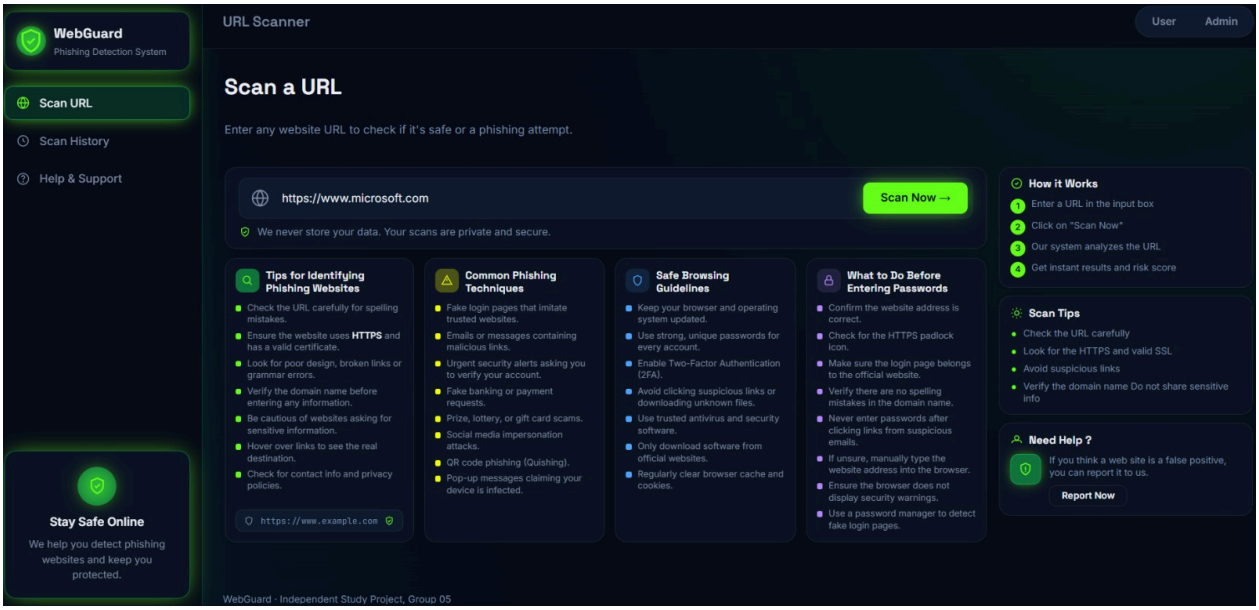


Figure 25: User Manual (Scan Page)

B.4 Detection Result Page

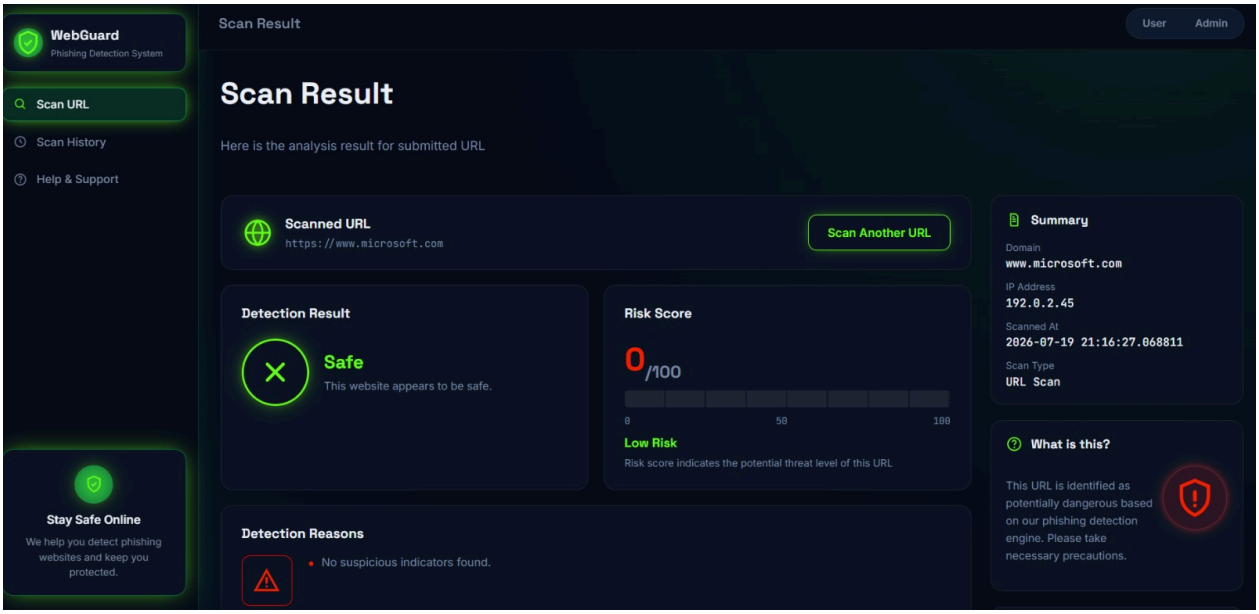


Figure 26: User Manual (Result Page)

The Detection Result page displays the analysis outcome, including the risk score, classification status (Safe, Suspicious, or Dangerous), and the reasons for the generated result. This helps users understand why a website is considered safe or potentially dangerous

B.5 Scan Analytics and History page

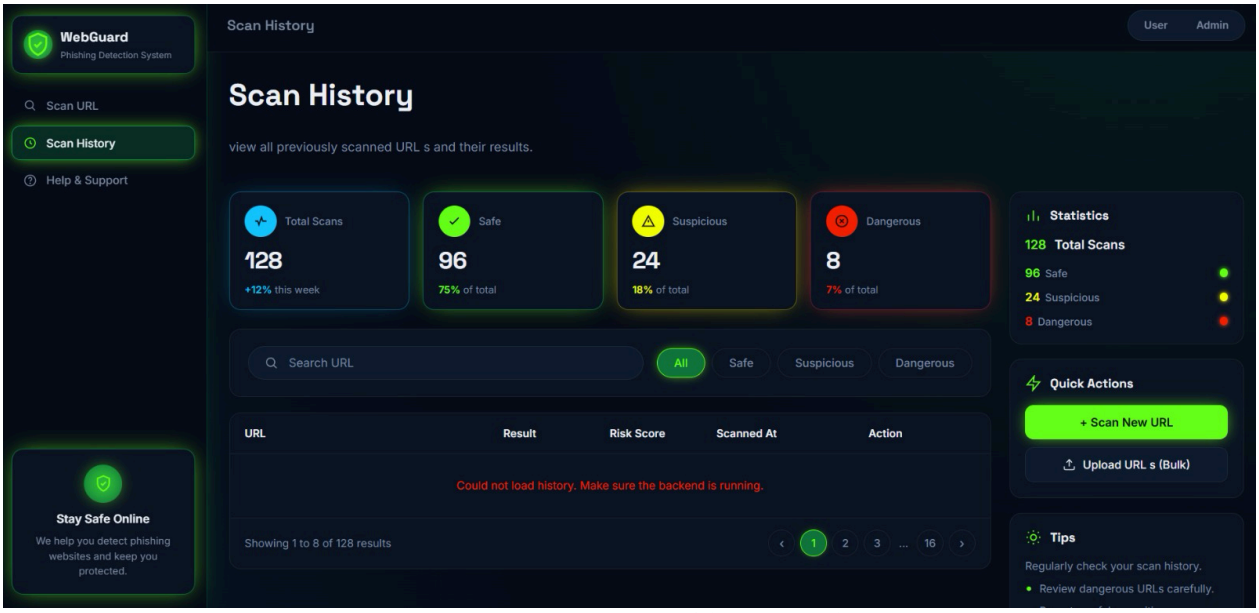


Figure 27: User Manual (Scan History)

The scan analytics page provides a summary of scanning activities, including the total number of scanned URLs, the number of safe ad dangerous websites , and recent scan activity. Graphical representations are used to help users understand scanning trends.
The scan History page allows users to view previously scanned URLs. For privacy and security few URLs being displayed. The page also shows the risk score, detection status and scan date

B.6 Browser Extension

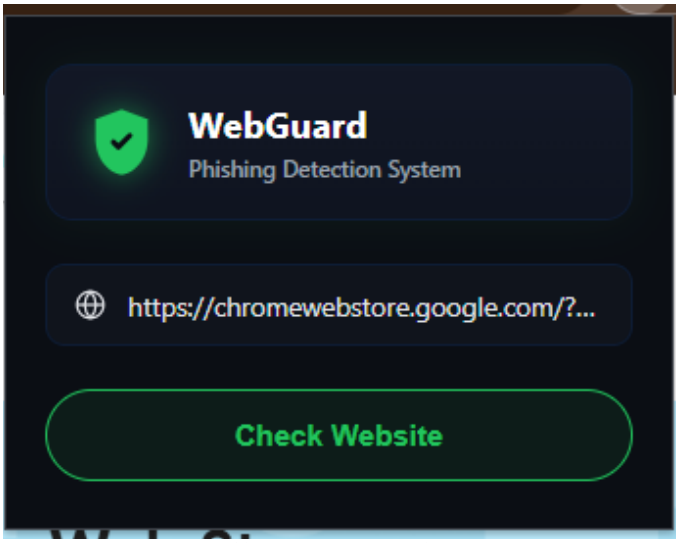


Figure 28: User Manual(Browser extension)

The browser extension enables users to scan the currently visited website directly from Google Chrome. When the Scan Current Website button is clicked, the extension captures the active URL and sends it to the backend server for analysis

B. 7 Administrator Login

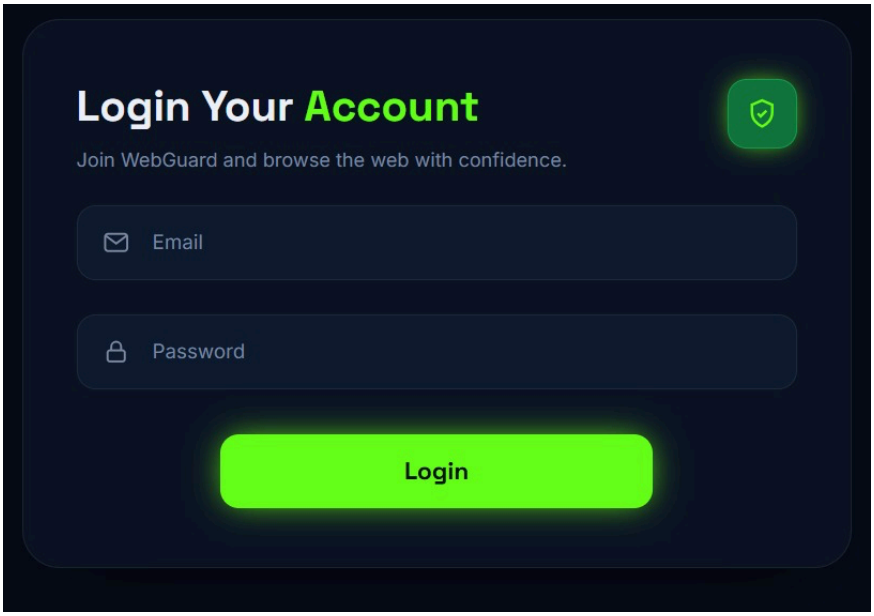


Figure 29: User manual (Admin login)

The Administrator Login page provides secure access to administrative functions. Authorized administrators can log in using their credentials to monitor system activities and view scanning statistics

B.8 Administrator Dashboard

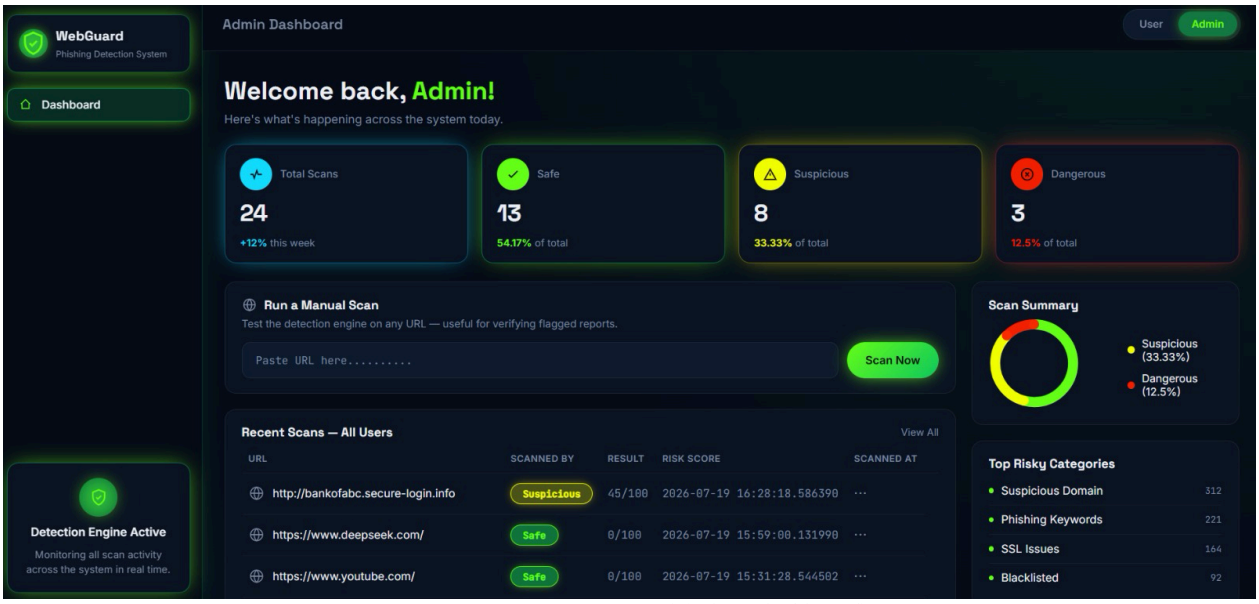


Figure 30: User Manual (Admin dashboard)

The Administrator Dashboard displays overall system statistics, recent scanning activities, and stored scan records. Administrators can use this page to monitor system usage and analyze detection results.

Appendix C: Important source codes

C.1 URL Detection Logic

This code check whether a URL contains suspicious characteristics such as excessive length, special characters, IP addresses, missing HTTPS, suspicious domain patterns and suspicious keywords. And show the reason.

```python
import re



def analyze_url(url):
    score = 0
    reasons = []


    if len(url) > 75:
        score += 20
        reasons.append("URL is very long")



    if re.search(r"\d+\.\d+\.\d+\.\d+", url):
        score += 30
        reasons.append("IP address used instead of domain name")


    keywords = ["login", "verify", "update", "secure", "free", "bank"]
    if any(word in url for word in keywords):
        score += 10
        reasons.append("Suspicious keyword found")


    special_chars = ["@" , "%", "$","&"]
    if any(char in url for char in special_chars):
        score += 10
        reasons.append("Suspicious special characters found")


    if not url.startswith("https"):
        score += 15
        reasons.append("URL is not using HTTPS")

    if "-" in url and any(word in url for word in keywords):
```

Figure 31: Detection logic source code

C.2 Risk Score Calculation

This code generates a risk score and classifies the website as Safe, Suspicious, or Dangerous based on the detected indicators.

Table 4: Score table

Score <= 30	Safe
Score <=60	Suspicious
Score > 60	Dangeous

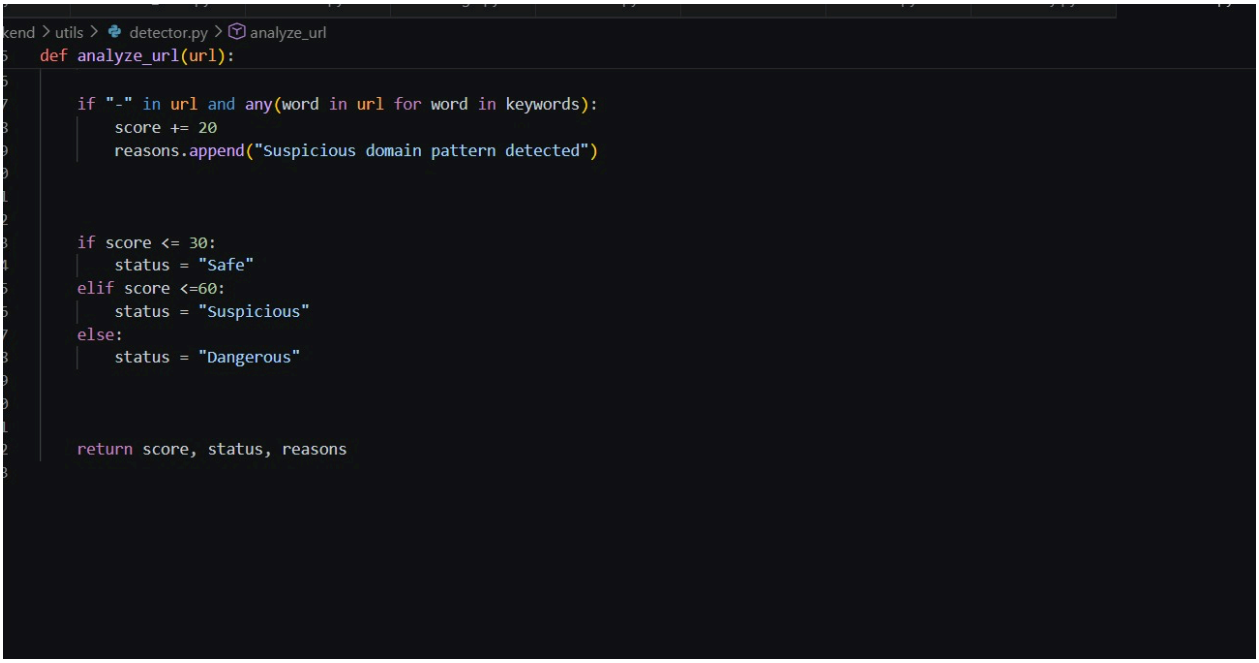


Figure 32: Risk score calculation source code

C.3 MongoDB Connection

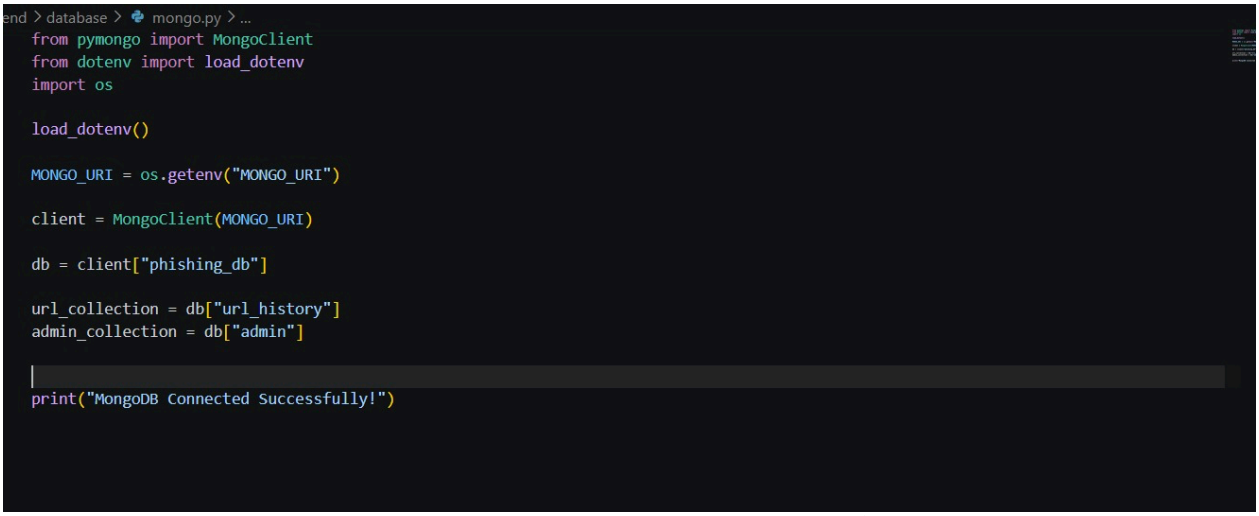


Figure 33: MongoDB Connection source code

This code establishes the connection between the python backend and the MongoDB database.

C.4 MongoDB database Collection

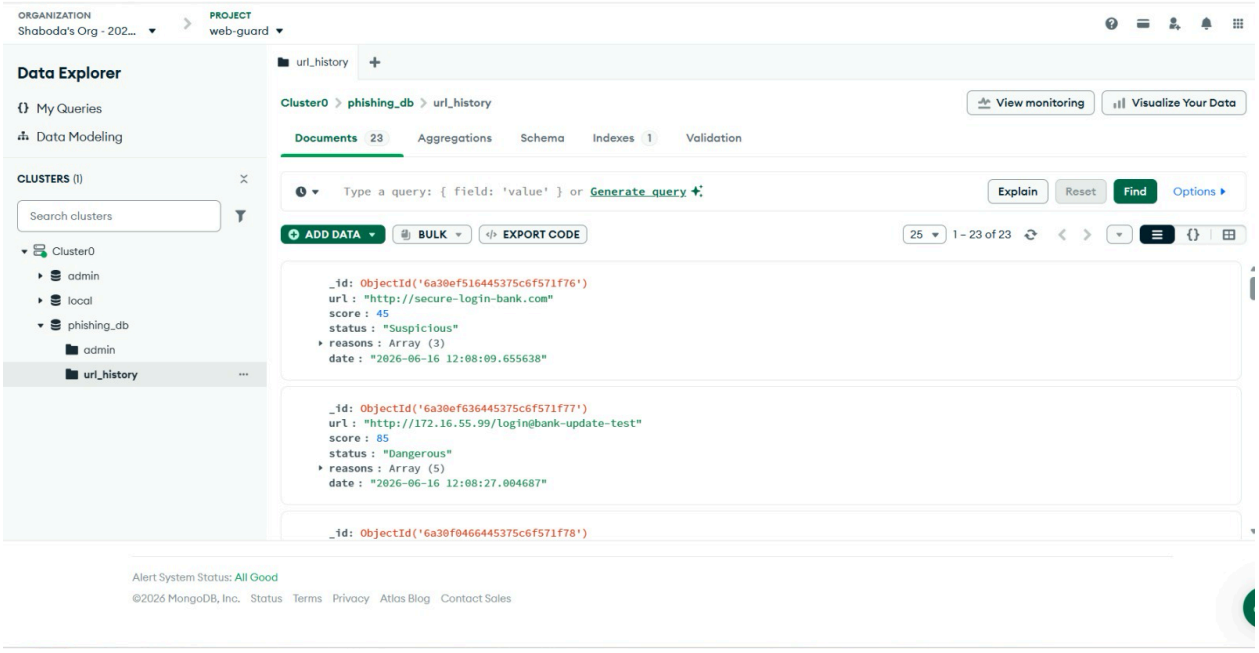


Figure 34: MongoDB collections

C.5 Browser Extension Manifest

This file defines the browser extension version, description , pop up page and host permission.

```
{
  "manifest_version": 3,
  "name": "WebGuard – Phishing Detection System",
  "version": "1.0",

    "icons": {

    "128": "icon/icon128.png"
  },

  "description": "Checks the current website for phishing indicators.",

  "permissions": [
    "tabs",
    "storage"
  ],

  "host_permissions": [
    "http://127.0.0.1:5000/*",
    "http://localhost:5000/*",
    "http://172.17.53.117:5000/*"
  ],

  "action": {
    "default_popup": "popup.html"
  }
}
```

Figure 35: Browser extension manifest source code

C.6 Browser extension Popup script

```
// Connect Flask Backend
async function checkUrlWithBackend(url) {

  console.log("Sending to Flask:", url);
  const response = await fetch("http://localhost:5000/check-url", {
    method: "POST",
    headers: {
      "Content-Type": "application/json"
    },
    body: JSON.stringify({ url: url })
  });

  if (!response.ok) {
    throw new Error("Backend Error");
  }

  const result = await response.json();

  console.log("Backend result:", result);

  if(result.error){
  throw new Error(result.error);
}
```

Figure 36: Browser extension popup source code

This script capture the current webpage URL and sends it to the backend server for analysis.

Appendix D: Test Result

This appendix represents sample test results of the phishing detection system. The tests were conducted using URLs with different risk levels verify that the system correctly classifies websites and generates appropriate risk score and explanation.

D.1 Safe URL test result

Table 5: Safe result table

Scanned URL	https://www.microsoft.com
Detection Result	Safe
Risk Score	0/100
Detection Result	No suspicious characters found

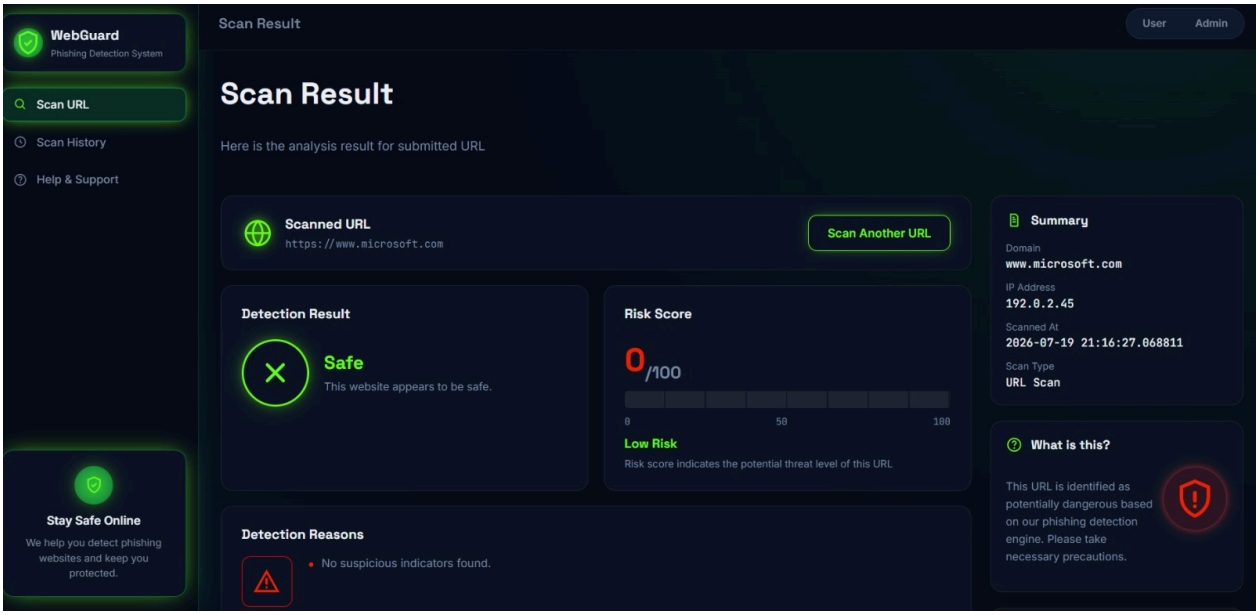


Figure 37: Safe result page

The system correctly identified the URL as safe because it uses HTTPS, contains a valid domain name, and does not contain suspicious keywords or patterns

D.2 Suspicious URL test result

Table 6: Suspicious result table

Scanned URL	https://amazon-verify-account.ne
Detection Result	Suspicious
Risk Score	30/100
Detection Result	• Suspicious keyword found • Suspicious domain pattern detected

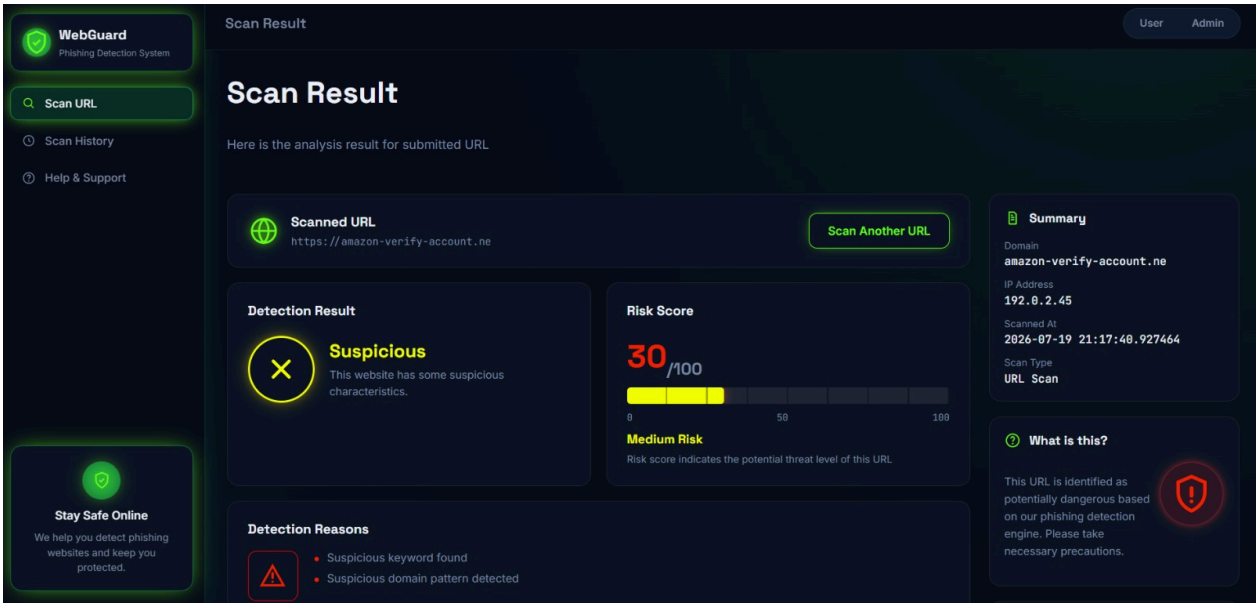


Figure 38: Suspicious result page

The system marked the URL as suspicious because it detect suspicious domain pattern and contains keyword that commonly associated with phishing attempts.

D.3 Dangerous URL test result

Figure 39: Dangerous result table

Scanned URL	https://secure-paypal-account-verifications-login-confire-.....
Detection Result	Dangerous
Risk Score	60/100
Detection Result	• Suspicious keyword found • Suspicious domain pattern detected • Long URL •

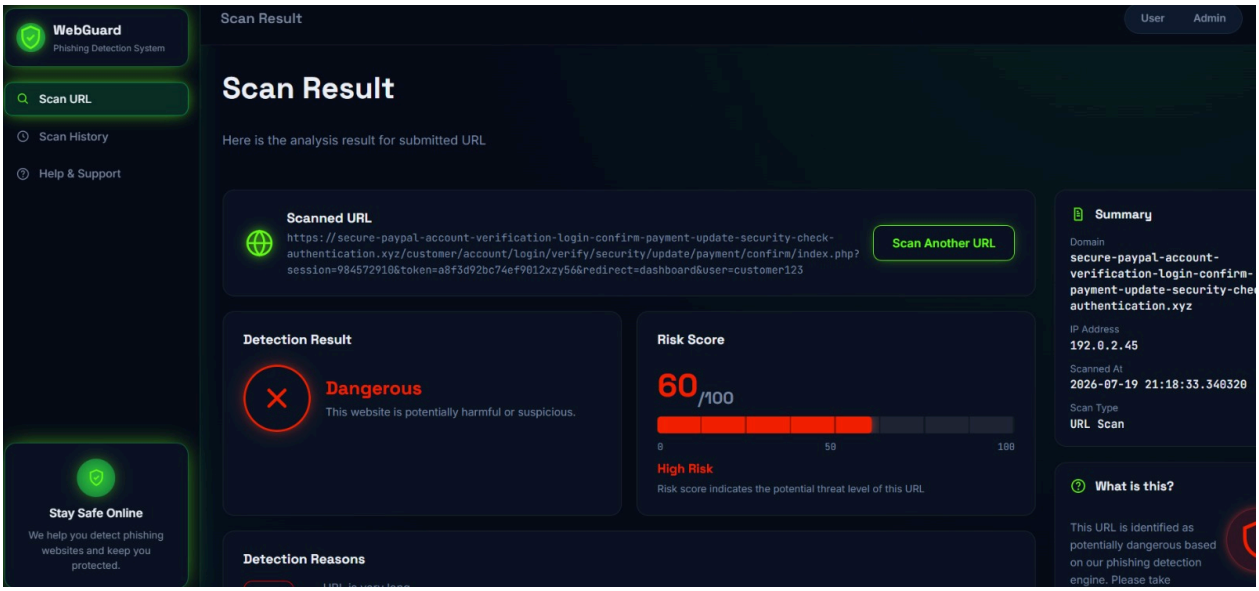


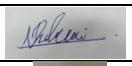
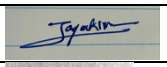
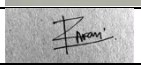
Figure 40: Dangerous Result page

The system marked the URL as suspicious because it detect suspicious domain pattern and contains keyword that commonly associated with phishing attempts.

Group Details

GROUP NO: [_ 05_]

Group Members

Registration Number	Name	Contact Number	Signature
UWU/ICT/23/004	K. A. N. P. Perera	**075 6812635**	
UWU/ICT/23/005	L. V. C. S. U. Kumari	**070 1593628**	
UWU/ICT/23/026	S. H. Thebuwana	**075 9983411**	
UWU/ICT/23/080	A. W. D. H. T. Jayawickrama	**071 5945792**	
UWU/ICT/23/081	S. A. N. Rathnasinghe	**076 0314524**	

Supervisor/s

Name	E-mail	Contact No
Mr r. V. Dishankan	dishankan@uwu.ac.lk	

Approval Signatures

Date :

Evaluation Panel Member

Date : Date :

[Name] [Name]
Project Supervisor Project Coordinator